



ASSEMBCAD: Production-Ready CAD Assembly Generation from Natural Language

Yurui Dong^{1,2*}, Shu Zou^{1,4,*}, Siqi Li^{1,5}, Nianchen Deng¹, Hongbin Zhou¹, Xuemeng Yang¹, Pinlong Cai¹, Licheng Wen^{1,3}, Xinyu Cai^{1†}, Botian Shi^{1,3}

¹ Shanghai Artificial Intelligence Laboratory, ² Fudan University, ³ Shanghai Innovation Institute

⁴ The Australian National University, ⁵ Zhejiang University

* Equal Contribution, † Corresponding Author

Recent advances in large language models and programmatic CAD have significantly improved Text-to-CAD generation for individual parts. However, production-ready mechanical assembly generation remains largely unsolved. Unlike single-part modeling, assemblies require coordinated reasoning over multiple components, functional interfaces, assembly relations, engineering principles, and physical consistency. Consequently, directly generating executable CAD code is insufficient for constructing mechanically valid and reusable assemblies.

We present ASSEMBCAD, an axiom-grounded framework for production-ready CAD assembly generation from natural language. Instead of representing an assembly as monolithic CAD code, ASSEMBCAD first constructs an axiomatic Assembly Specification consisting of typed parts, geometry-backed parts, executable mates, and engineering axioms. Each assembly relation is explicitly grounded in one or more engineering principles, making the resulting specification interpretable, reusable, and verifiable. To realize this specification, ASSEMBCAD introduces a port- and mate-based CAD assembly library that executes symbolic assembly relations through deterministic mate transformations and validates declared interfaces using concrete B-Rep geometric evidence.

Built on this representation and library, ASSEMBCAD further supports on-demand synthesis of reusable parametric component factories for both standard and open-world geometries. A deterministic verification pipeline then checks interface validity, clash consistency, graph connectivity, degree-of-freedom constraints, and engineering-rule compliance, producing production-ready CAD assemblies together with interpretable verification reports. Experiments on ASSEMBENCH show that ASSEMBCAD substantially improves assembly preservation and physical validity over code-centric CAD generation baselines, while generalizing across different foundation-model backbones. By combining axiom-grounded assembly reasoning with deterministic geometric execution, ASSEMBCAD extends Text-to-CAD from isolated part generation toward production-ready mechanical assembly design.



Date: July 7, 2026



AssemCAD：基于自然语言的生产级CAD 装配生成系统

董玉瑞^{1,2*}、邹书^{1,4,*}、李思琪^{1,5}、邓念辰¹、周洪斌¹、杨雪萌¹、蔡品龙¹、文立成^{1,3}、蔡新宇^{1†}、史博天^{1,3}

¹ 上海人工智能实验室, ² 复旦大学, ³ 上海创新研究院

⁴ 澳大利亚国立大学, ⁵ 浙江大学

* 同等贡献, † 通讯作者

近年来, 大型语言模型与程序化CAD技术的进展显著提升了针对单个零件的文本到CAD生成能力。然而, 实现可直接投入生产的机械装配体生成问题在很大程度上仍未得到解决。与单零件建模不同, 装配体需要对多个部件、功能接口、装配关系、工程原理及物理一致性进行协同推理。因此, 直接生成可执行的CAD代码不足以构建机械上有效且可复用的装配体。

我们提出了AssemCAD——一个基于公理的框架, 可从自然语言生成生产就绪的CAD装配体。与将装配体表示为单一的CAD代码不同, AssemCAD首先构建一个公理化的装配规范, 该规范由类型化零件、基于几何的端口、可执行配对关系及工程公理组成。每个装配关系均明确依据一个或多个工程原则, 从而使生成的规范具备可解释性、可复用性和可验证性。为实现这一规范, AssemCAD引入了一个基于端口和配对关系的CAD装配库, 该库通过确定性的配对变换执行符号化装配关系, 并利用具体的B-Rep几何证据验证声明的接口。

基于这一表示方法和库, assemcad进一步支持针对标准几何与开放世界几何的可复用参数化组件工厂的按需合成。随后, 一个确定性验证流程会检查接口有效性、冲突一致性、图连通性、自由度约束及工程规则合规性, 从而生成可直接用于生产的CAD装配体, 并附带可解释的验证报告。在assembench上进行的实验表明, assemcad相较于以代码为中心的CAD生成基线, 在装配保真度和物理有效性方面均取得显著提升, 且能在不同基础模型骨干架构间实现泛化。通过将公理导向的装配推理与确定性几何执行相结合, assemcad将文本到CAD的流程从孤立的零件生成扩展至可直接用于生产的机械装配设计领域。

 日期: 2026年7月7日

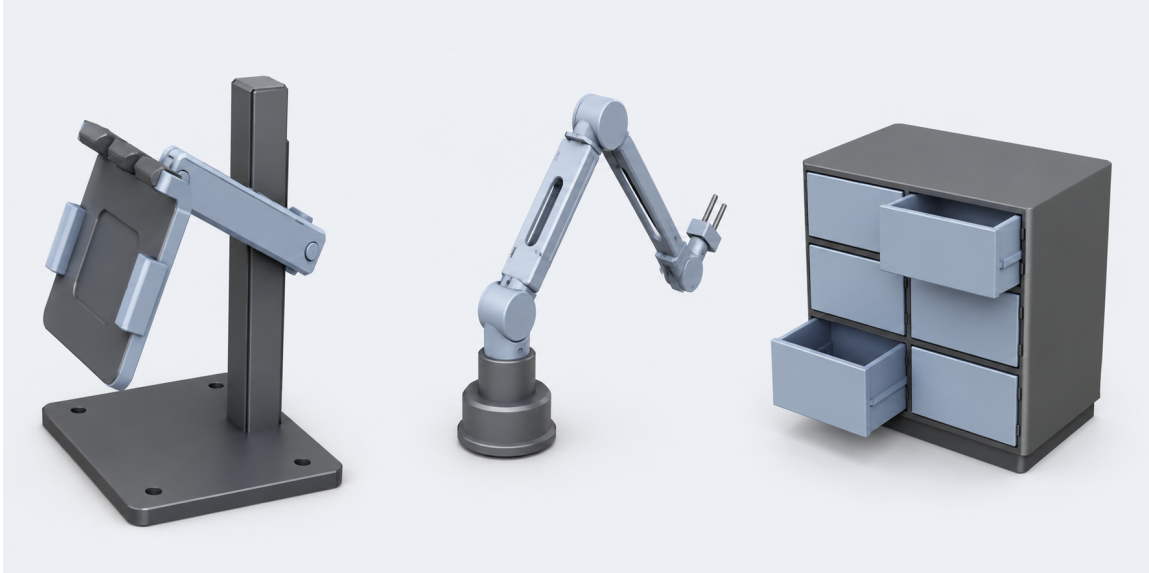


Figure 1: Representative production-ready CAD assemblies generated directly from natural language. **ASSEMBCAD** generates mechanically executable assemblies with valid assembly structures and articulated joints, rather than merely static CAD geometry.

1 Introduction

Computer-aided design (CAD) [1] models are fundamental to modern engineering workflows, supporting downstream applications such as manufacturing, simulation, and assembly planning. While traditional CAD systems rely on interactive commercial software (e.g., SolidWorks) [2, 3, 4], where design intent is implicitly encoded in GUI operations and feature histories, programmatic CAD frameworks such as CadQuery and OpenSCAD [5, 6] provide explicit, parametric, and editable representations that naturally support procedural modeling and automated reasoning [7, 8, 9, 10, 11, 12]. Combined with recent advances in large language models (LLMs), substantial progress has been made in generating executable CAD models from natural language and visual inputs. Representative methods such as Text2CAD [13], PointCloud2CAD [14], and Cadrille [15] demonstrate strong capabilities for text-to-CAD, image-to-CAD, and multimodal CAD generation, while frontier code-centric foundation models (e.g., Claude and Codex [16, 17, 18, 19, 20]) further enable zero-shot executable CAD program synthesis. Together, these advances establish natural-language-driven CAD generation as a promising paradigm for automated engineering design.

Despite these advances, as illustrated in Figure 2, production-ready¹ CAD assembly generation remains fundamentally more challenging than isolated part synthesis. Unlike standalone parts, mechanical assemblies are structured systems whose functionality depends not only on individual components but also on their interactions. This introduces three fundamental challenges. First, assemblies require explicit reasoning over functional relationships and geometric constraints across multiple components, rather than generating independent geometries. Second, the component space is inherently open-ended, requiring support for both standard parts and application-specific geometries beyond predefined templates. Third, correctness extends beyond executable CAD code or individually valid solids. Production-ready assemblies additionally require physically realizable interfaces, correct assembly relations, and interference-free configurations. These challenges fundamentally distinguish assembly generation from single-part CAD modeling and explain the gap between the left and right examples in Figure 2.

These challenges suggest that production-ready CAD assembly generation should be viewed as an engineering pipeline rather than a code generation problem. As illustrated in Figure 2, producing executable CAD code alone is insufficient, as failures may still arise from invalid components, incorrect assembly relations, or inconsistent assembly configurations. We therefore decompose the problem into three complementary stages, where foundation models perform semantic reasoning and structured planning, while programmatic CAD kernels provide deterministic geometry construction and physical verification. Specifically, the pipeline addresses three fundamental questions: what to build, how to build it, and how to verify the resulting assembly. This decomposition enables stage-wise validation and reliable production-ready assembly generation.

¹In this work, production-ready refers to assemblies that are executable, structurally connected, interface-valid, and free of unintended geometric interference, rather than fully optimized for manufacturing processes, materials, or tolerance analysis.



图1：直接从自然语言生成的代表性可生产CAD装配体。**assemcad**能够生成具备有效装配结构和活动关节的机械可执行装配体，而非仅生成静态CAD几何图形。

1 引言

计算机辅助设计（CAD）[1]模型是现代工程工作流程的基础，可支持制造、仿真和装配规划等下游应用。传统CAD系统依赖于交互式商业软件（如SolidWorks）[2, 3, 4]——其中设计意图隐含在GUI操作和特征历史中；而程序化CAD框架（如CadQuery和OpenSCAD）[5, 6]则提供了显式、参数化且可编辑的表示方式，能自然支持过程化建模与自动化推理[7, 8, 9, 10, 11, 12]。结合大语言模型（LLMs）的最新进展，从自然语言和视觉输入生成可执行CAD模型的研究已取得显著突破。代表性方法如Text2CAD[13]、PointCloud2CAD[14]和Cadrille[15]，在文本到CAD、图像到CAD及多模态CAD生成方面展现出强大能力；而前沿的代码中心基础模型（如Claude和Codex[16, 17, 18, 19, 20]）则进一步实现了零样本可执行CAD程序合成。这些进展共同确立了基于自然语言的CAD生成作为自动化工程设计的一种极具前景的范式。尽管取得了这些进展，如图2所示，可直接用于生产的¹CAD装配生成在本质上仍比孤立的零件合成更具挑战性。与独立零件不同，机械装配体是结构化系统，其功能不仅取决于单个组件，还取决于各组件之间的相互作用。这带来了三个根本性挑战：首先，装配体需要对跨多个组件的功能关系和几何约束进行显式推理，而非仅生成独立的几何形状；其次，组件空间本质上是开放的，需支持标准零件及超越预定义模板的应用特定几何形状；第三，正确性不仅限于可执行的CAD代码或单独有效的实体模型——可直接用于生产的装配体还需具备物理可实现的接口、正确的装配关系以及无干涉的配置。这些挑战从根本上将装配生成与单零件CAD建模区分开来。并未解释图2中左右两例之间的差异。

这些挑战表明，生产就绪的CAD装配生成应被视为一个工程流程，而非代码生成问题。如图2所示，仅生成可执行的CAD代码是不够的，因为仍可能因无效组件、错误的装配关系或不一致的装配配置而出现故障。因此，我们将该问题分解为三个互补阶段：基础模型负责进行语义推理和结构化规划，而程序化CAD内核则提供确定性的几何构造与物理验证。具体而言，该流程解决了三个根本性问题：应构建什么、如何构建以及如何验证最终生成的装配体。这种分解方式使得各阶段均可进行验证，并能实现可靠的生产就绪装配生成。

¹在本研究中，“生产就绪”一词指的是可执行、结构连接正确、接口有效且无意外几何干涉的装配体，而非针对制造工艺、材料或公差分析进行了全面优化的装配体。

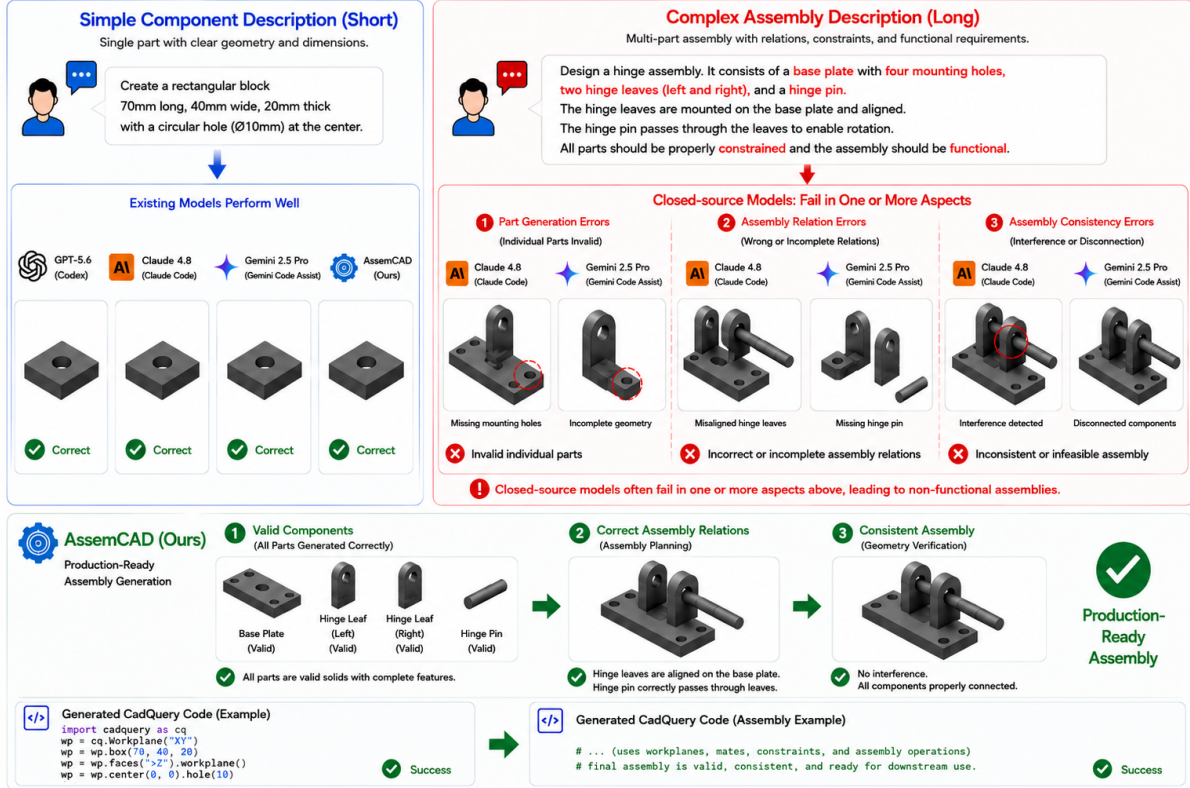


Figure 2: Motivation and overview of ASSEMCAD. Existing foundation models perform well on single-part CAD generation but struggle with production-ready assembly generation, where failures commonly arise from invalid components, incorrect assembly relations, and inconsistent assembly configurations. To address these challenges, ASSEMCAD decomposes assembly generation into three stages: structured assembly specification, on-demand component synthesis, and geometry-aware verification, enabling the generation of production-ready CAD assemblies from natural-language specifications.

Guided by this insight, we present ASSEMCAD, a framework for production-ready CAD assembly generation from natural-language specifications. ASSEMCAD combines foundation models with programmatic CAD kernels to bridge semantic reasoning and deterministic geometric execution. Corresponding to the three stages of assembly generation, the framework comprises (1) a structured assembly specification for representing components, interfaces, and assembly semantics, (2) an on-demand component synthesis module for generating reusable parametric parts, and (3) a geometry-aware verification module for ensuring physical consistency and assembly correctness. Together, these components enable reliable production-ready CAD assembly generation through a transparent and verifiable engineering pipeline.

Our contributions are four-fold:

- We present ASSEMCAD, a framework for production-ready CAD assembly generation from natural language. It formulates assembly generation as the construction of verifiable engineering specifications rather than direct CAD code generation.
- We introduce an axiomatic assembly specification that explicitly represents typed parts, geometry-backed ports, executable mates, and engineering axioms. By grounding each assembly relation in explicit engineering principles, the specification provides an interpretable bridge between natural-language intent and geometric realization.
- We develop a port- and mate-based CAD assembly library that operationalizes the specification through reusable assembly-ready components, deterministic mate transforms, and port-geometry consistency checks. This library supports both standard components and open-world synthesized parts while ensuring that declared interfaces are physically meaningful.
- We establish ASSEMBENCH together with an assembly-centric evaluation protocol, and demonstrate through extensive experiments that ASSEMCAD substantially improves assembly preservation, physical validity, and generalization across different foundation models.

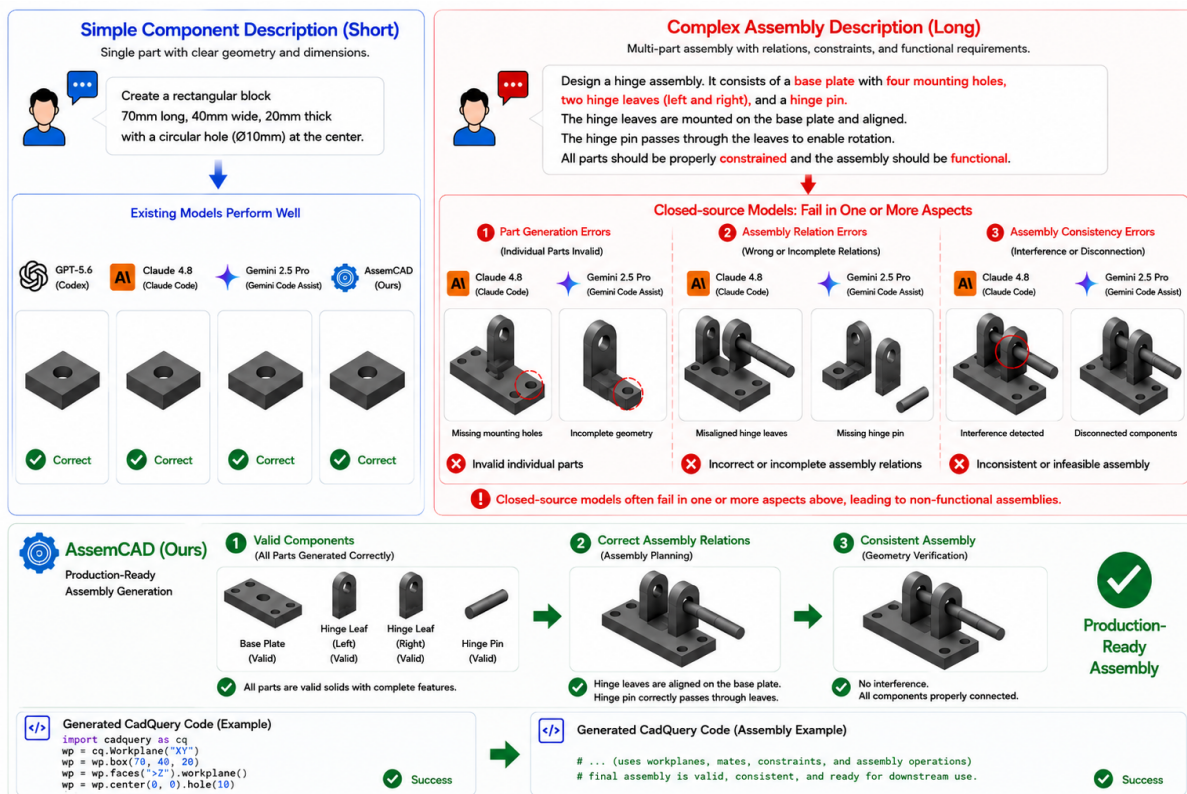


图2: assemcad的动机与概述。现有基础模型在单部件CAD生成任务中表现良好，但在生产级装配生成方面存在困难——其失败通常源于无效组件、错误的装配关系及不一致的装配配置。为应对这些挑战，assemcad将装配生成过程分解为三个阶段：结构化装配规范制定、按需组件合成以及几何感知验证，从而能够根据自然语言规格生成符合生产标准的CAD装配体。

基于这一洞见，我们提出了AssemCAD——一个能够从自然语言规格生成可直接投产的CAD装配方案的框架。AssemCAD将基础模型与程序化CAD内核相结合，实现了语义推理与确定性几何执行之间的衔接。该框架对应装配生成的三个阶段，具体包含：(1) 用于表示组件、接口及装配语义的结构化装配规范；(2) 用于生成可复用参数化零件的按需组件合成模块；(3) 用于确保物理一致性与装配正确性的几何感知验证模块。这些组件共同通过一条透明且可验证的工程流程，实现了可靠的可直接投产CAD装配方案生成。

我们的贡献可归纳为四个方面：

- 我们提出了AssemCAD——一个能够从自然语言生成生产级CAD装配的框架。该框架将装配生成问题表述为构建可验证工程规范的过程，而非直接进行CAD代码生成。
- 我们引入了一种公理化装配规范，该规范能够明确表示带类型标注的部件、基于几何结构的端口、可执行匹配项以及工程公理。通过将每种装配关系建立在明确的工程原则之上，该规范为自然语言意图与几何实现之间搭建了一个可解释的桥梁。
- 我们开发了一个基于端口与配对的CAD装配库，该库通过可复用的装配就绪组件、确定性配对变换及端口几何一致性检查来实现规格说明。该库既支持标准组件也支持开放世界合成零件，同时确保声明的接口具有物理意义。
- 我们与Assembench共同建立了以装配为中心的评估协议，并通过大量实验表明，AssemCAD能显著提升装配保真度、物理有效性以及在不同基础模型间的泛化能力。

2 Related Work

2.1 Programmatic CAD Generation and B-Rep Learning

Recent work on generative CAD has explored multiple representations, including construction sequences, executable CAD programs, and boundary representations. Sequence-based methods [3, 21, 22] model CAD construction histories autoregressively, while Fusion 360 Gallery [23] provides human-authored sketch-and-extrude programs and an interactive reconstruction environment, and Free2CAD [24] parses freehand drawings into CAD command sequences. Text-conditioned and multimodal methods [25, 26] further extend this paradigm to natural-language [7, 13, 27], image [15], and point-cloud inputs [14, 28], producing editable CAD programs or command sequences. In parallel, B-Rep representation learning methods such as UV-Net [29] and BRepNet [2] operate directly on parametric surfaces and CAD topology, while B-Rep reconstruction and generative models such as ComplexGen [30], SolidGen [31], and BrepGen [32] synthesize boundary representations without relying on construction-history supervision. These works establish strong foundations for part-level CAD generation and reconstruction. However, they primarily focus on individual solids or isolated construction histories, and do not explicitly model inter-part assembly relations, functional interfaces, or physical consistency required by production-ready assemblies.

2.2 CAD Assembly Modeling and Relation Inference

Mechanical assemblies are not merely collections of valid parts; their functionality depends on mates, contacts, kinematic constraints, and assembly topology [33]. Classical constraint-based assembly modeling studies represent assemblies through geometric mating constraints and reason about under-, over-, and fully-constrained configurations, interference, and remaining degrees of freedom [34, 35]. Recent learning-based methods revisit these problems using CAD data. JoinABLE [36] predicts parametric CAD joints from solid models and releases assembly data with joints, contact surfaces, holes, and assembly graphs, while Mates2Motion [37] learns degrees of freedom between mated CAD parts. Beyond joint prediction, previous work also investigates B-Rep-based automatic mate prediction [33] and physically feasible assembly planning, often through assembly-by-disassembly [38, 39]. These methods typically assume that component geometries are already given and focus on recovering or predicting assembly relations. In contrast, AssemCAD generates both open-world components and their assembly specification from natural language, and verifies the resulting structure through deterministic geometric execution.

2.3 Foundation Models and Agentic CAD Systems

Recent foundation models [16, 17, 18, 19, 20] have demonstrated remarkable capabilities in code generation, spatial reasoning, and long-horizon planning, while agentic frameworks further improve reliability through iterative tool use, execution feedback, external memories, and reusable skills. These advances have naturally extended to CAD generation [26]. Cad-assistant [40] formulates CAD interaction as a generic tool-augmented task-solving problem, where a vision-language model plans CAD-specific actions that are executed through FreeCAD and Python APIs. CADSmith [9] improves text-to-CadQuery generation with a multi-agent pipeline and programmatic geometric validation, combining execution repair with CAD-kernel measurements such as bounding boxes, volumes, and solid validity. Solver-aided systems such as AIDL [41] further offload low-level spatial reasoning to geometric constraint solvers through a hierarchical domain-specific language.

These systems demonstrate the promise of tool-augmented, execution-grounded, and solver-aided foundation models for CAD generation. However, their reliability is mainly achieved through local code repair, geometric validation, or solver-assisted construction. In contrast, AssemCAD improves practical reliability by coupling an assembly-oriented component library, a curated engineering axiom system, and a deterministic verification pipeline. This design grounds generation in reusable components and explicit port-mate semantics, while verifying clashes, connectivity, degrees of freedom, and assembly consistency, enabling more robust production-ready mechanical assembly generation.

Overall, prior work has advanced CAD generation, assembly modeling, and foundation-model-driven CAD systems from complementary perspectives. Programmatic and B-Rep methods improve part-level CAD synthesis, assembly modeling methods recover joints, mates, and motions from existing geometries, while agentic CAD systems enhance executability through tool use, repair, validation, or solver grounding. However, production-ready CAD design requires more than geometric plausibility. Prior work on knowledge-based engineering, ontology-based assembly design, and semantic mates shows that assembly relations carry engineering meaning beyond raw spatial transformations. Inspired by this line of work, AssemCAD associates each mate with engineering axioms covering foundations, constraints, kinematics, bearings, gears, fastening, hinges, and related categories. Unlike conventional knowledge-based systems that mainly support retrieval or post-hoc checking, AssemCAD integrates engineering semantics into the generative loop: axioms guide semantic decomposition, constrain component synthesis, and are propagated to the final verification report. Together with an assembly-oriented component library and deterministic

2 相关工作

2.1 程序化CAD生成与B-Rep学习

近期生成式CAD研究探索了多种表示方法，包括构造序列、可执行CAD程序及边界表示。基于序列的方法[3, 21, 22]通过自回归方式对CAD构造历史进行建模；而Fusion 360 Gallery[23]则提供人工绘制的草图-拉伸程序及交互式重建环境；Free2CAD[24]可将自由手绘图纸解析为CAD命令序列。文本条件与多模态方法[25, 26]进一步扩展了这一范式，支持自然语言[7, 13, 27]、图像[15]及点云输入[14, 28]，并可生成可编辑的CAD程序或命令序列。与此同时，B-Rep表示学习方法如UV-Net[29]和BRepNet[2]直接作用于参数化曲面与CAD拓扑结构；而B-Rep重建与生成模型如ComplexGen[30]、SolidGen[31]及BrepGen[32]则无需依赖构造历史监督即可合成边界表示。这些研究为零件级CAD生成与重建奠定了坚实基础。然而，它们主要关注单个实体或孤立的构建历史，未能明确建模零件间的装配关系、功能接口或生产级装配所需的物理一致性。

2.2 CAD装配建模与关系推理

机械装配体并非仅仅是有效零件的简单集合；其功能取决于配合件、接触点、运动学约束及装配拓扑结构[33]。传统的基于约束的装配建模方法通过几何配合约束来表征装配体，并对欠约束、过约束及完全约束配置、干涉情况以及剩余自由度进行推理[34, 35]。近期基于学习的方法利用CAD数据重新探讨了这些问题：JoinABLE[36]可从实体模型预测参数化CAD关节，并生成包含关节、接触面、孔洞及装配图谱的装配数据；而Mates2Motion[37]则可学习已配对CAD部件之间的自由度。除关节预测外，先前研究还探索了基于B-Rep的自动配合预测[33]及物理可行的装配规划——后者常通过拆卸-装配流程实现[38, 39]。这些方法通常假设零部件几何形状已知，并专注于恢复或预测装配关系。相比之下，AssemCAD能够从自然语言生成开放世界组件及其装配规范，并通过确定性几何执行验证生成的结构。

2.3 基础模型与智能体驱动的CAD系统

近期的基础模型[16, 17, 18, 19, 20]在代码生成、空间推理和长时程规划方面展现出卓越能力，而智能体框架则通过迭代工具使用、执行反馈、外部记忆及可复用技能进一步提升了可靠性。这些进展自然延伸至CAD生成领域[26]。Cad-Assistant[40]将CAD交互建模为一种通用的工具增强型任务求解问题——其中，视觉-语言模型会规划针对CAD的特定操作，并通过FreeCAD和Python API执行这些操作。CADSmith[9]借助多智能体流水线和程序化几何验证技术，改进了文本到CADQuery的生成过程，将执行修复与CAD内核测量指标（如边界框、体积及实体有效性）相结合。诸如AIDL[41]等求解器辅助系统，则通过分层的领域专用语言，将低级空间推理任务卸载至几何约束求解器。

这些系统展示了工具增强型、执行导向型及求解器辅助型基础模型在CAD生成领域的潜力。然而，其可靠性主要依赖于局部代码修复、几何验证或求解器辅助构建来实现。相比之下，AssemCAD通过耦合装配导向的组件库、精心编排的工程公理系统以及确定性验证流程，提升了实际可靠性。该设计将生成过程锚定于可复用组件与显式端口匹配语义之上，同时对碰撞、连通性、自由度及装配一致性进行验证，从而能够生成更稳健的生产级机械装配方案。

总体而言，先前的研究从互补的角度推进了CAD生成、装配建模及基础模型驱动的CAD系统发展。程序化方法与B-Rep方法可提升零件级CAD合成能力；装配建模方法能够从现有几何结构中恢复关节、配对关系及运动轨迹；而智能体驱动的CAD系统则通过工具使用、修复、验证或求解器接地等方式增强可执行性。然而，生产级CAD设计不仅需要几何合理性，还需具备更深层次的能力。此前关于知识型工程、本体化装配设计及语义配对的研究表明，装配关系蕴含着超越原始空间变换的工程意义。受这一研究路径启发，AssemCAD将每个配对关系与涵盖基础理论、约束条件、运动学、轴承、齿轮、紧固件、铰链及相关类别的工程公理相关联。与主要支持检索或事后校验的传统知识型系统不同，AssemCAD将工程语义整合至生成循环之中：这些公理指导语义分解过程，约束组件合成过程，并最终传递至最终验证报

verification pipeline, this enables AssemCAD to generate more reliable production-ready mechanical assemblies from natural language.

3 Problem Formulation

Traditional CAD Generation. Existing CAD generation methods typically formulate the task as directly translating textual or visual descriptions into geometric representations or executable CAD programs [13, 15]. Such formulations are well suited for individual part generation, where the output is a standalone geometry represented by a procedural CAD program or a sequence of construction operations. However, production-ready mechanical assemblies consist of multiple interacting components whose correctness depends not only on geometry, but also on interfaces, assembly relationships, and engineering semantics. In this work, we formulate production-ready CAD assembly generation as the generation of both a structured assembly specification and its corresponding geometric realization, enabling deterministic construction and engineering-aware verification.

3.1 Assembly Formulation

Core Formulation

Production-ready CAD assembly generation should not directly map natural-language descriptions to final geometry. Instead, it should first construct an intermediate representation that explicitly captures **components, interfaces, assembly relationships, and engineering semantics**. This intermediate representation enables **stage-wise construction** and **geometry-aware verification**, providing a principled bridge between semantic specifications and geometric realization.

Following this formulation, we introduce an intermediate representation, termed *Assembly Specification*, which serves as the bridge between semantic descriptions and geometric realization.

Definition 1 (Assembly Specification). *An assembly specification is defined as a triplet*

$$\mathcal{S} = (\mathcal{P}, \mathcal{M}, \mathcal{A}),$$

where $\mathcal{P}$ denotes a set of typed parts, $\mathcal{M}$ denotes a set of typed mates describing assembly relationships, and $\mathcal{A}$ denotes the engineering semantics associated with these relationships.

- $\mathcal{P} = \{p_1, \dots, p_n\}$ is a set of typed parts, where each part

$$p_i = (i d_i, f_i, \theta_i, \Pi_i)$$

consists of a factory $f_i \in \mathcal{F}$, its parameters θ_i , and a set of ports Π_i .

- $\mathcal{M} = \{m_1, \dots, m_k\}$ is a set of typed mates, where each mate

$$m_j = (\tau_j, b_j, c_j, \omega_j)$$

specifies a mate type $\tau_j \in \mathcal{T}$, two endpoints, and optional mate parameters ω_j .

- $\mathcal{A} \subseteq \mathcal{A}_{\text{axioms}}$ denotes the subset of engineering axioms that justify the assembly relationships.

Specifically, $\mathcal{A}$ captures the engineering semantics associated with assembly relationships. Its construction and role are described in the following subsection.

3.2 Engineering Semantics

Unlike conventional CAD systems that rely solely on geometric constraints, production-ready assemblies must also satisfy engineering semantics. For example, meshing gears require compatible transmission parameters, bearings require coaxial alignment, and fastening components must satisfy assembly constraints beyond pure geometry. To explicitly encode such knowledge, ASSEM-CAD introduces an engineering knowledge base consisting of 62 engineering axioms organized into 10 mutually exclusive and collectively exhaustive (MECE) categories, summarized in Table 1. Of these, 41 are distilled from assembly textbooks through a three-phase LLM-assisted extraction pipeline, and 21 are synthesized to cover component families present in the CAD libraries but absent from the source corpus; the complete extraction methodology is detailed in Appendix A.

Table 1: Engineering axiom categories ($|\mathcal{A}_{\text{axioms}}| = 62$, curated from 139 candidates).

Code	Category	n	Code	Category	n
F	Foundations	5	N	Structural	7
C	Constraints	5	P	Power/Shafts	3
K	Kinematic	5	S	Sequencing	6
B	Bearings	4	T	Fastening	12
G	Gears	9	H	Hinges	6

告中。结合面向装配的组件库与确定性验证流程，AssemCAD能够从自然语言生成更可靠的、可直接投入生产的机械装配方案。

3 问题表述

传统CAD生成。现有的CAD生成方法通常将该任务表述为直接将文本或视觉描述翻译为几何表示或可执行的CAD程序[13, 15]。此类表述方式非常适合单个零件生成场景，其输出结果是通过过程化CAD程序或一系列构造操作来表示的独立几何模型。然而，生产级机械装配体由多个相互作用的部件组成，其正确性不仅取决于几何结构，还取决于接口、装配关系及工程语义。在本研究中，我们将生产级CAD装配体生成问题表述为同时生成结构化的装配规范及其对应的几何实现，从而能够实现确定性的构造与工程感知验证。

3.1 装配配方

核心处方

生产级CAD装配生成不应直接将自然语言描述映射至最终几何模型，而应首先构建一个中间表示——该表示能明确捕捉**组件、接口、装配关系及工程语义**。此中间表示可支持**分阶段构建与几何感知验证**，从而在语义规范与几何实现之间架设一条基于原理的桥梁。

基于这一表述框架，我们引入了一种中间表示形式——**组装规范**，其充当了语义描述与几何实现之间的桥梁。

定义1（装配规范）：装配规范定义为一个三元组

$$S = (P, M, A),$$

其中， P 表示一组带类型标注的零件， M 表示描述装配关系的带类型标注的配合关系集合，而 A 则表示与这些关系相关的工程语义。

- $P = \{p_1, \dots, p_n\}$ 是一个带类型标注的部件集合，其中每个部件

$$p_i = (i d_i, f_i, \theta_i, \Pi_i)$$

由工厂 $f_i \in F$ 、其参数 θ_i 以及一组端口 Π_i 组成。

- $M = \{m_1, \dots, m_k\}$ 是一个带类型标记的配对集合，其中每个配对

$$m_j = (\tau_j, b_j, c_j, \omega_j)$$

指定配对类型 $\tau_j \in T$ 、两个端点以及可选的配对参数 ω_j 。

- $A \subseteq A_{\text{axioms}}$ 表示能够证明装配关系的工程公理子集。

具体而言， A 可用于捕捉与装配关系相关的工程语义；其构建方式及作用将在下一个小节中予以阐述。

3.2 工程语义学

与仅依赖地理信息的传统CAD系统不同，在满足几何约束的前提下，可量产的组件还必须满足工程语义。例如，网格齿轮传动系统需具备兼容的传动参数。轴承需进行同轴对齐，且紧固件组件必须满足除纯装配约束之外的其他装配约束。几何学。为显式编码此类知识，ASSEMCAD介绍了一个工程知识库，由62条工程公理组成，这些公理共分为10类。互斥且穷尽（MECE）

表1：工程公理类别（ $|A_{\text{axioms}}| = 62$ ，从139个候选项中精选）

编码	类别	n	编码	类别	n
F	基础部分	5	N	结构的	7
C	约束条件	5	P	动力/轴系	3
K	运动学	5	S	测序	6
B	轴承	4	T	紧固件	12
G	齿轮	9	H	铰链	6

各类别汇总于表1中。其中，41个类别通过三阶段LLM辅助提取流程从装配教科书中提炼得出，另有21个类别为合成生成，旨在涵盖CAD库中存在的但源语料库中缺失的组件家族；完整的提取方法详见附录A。

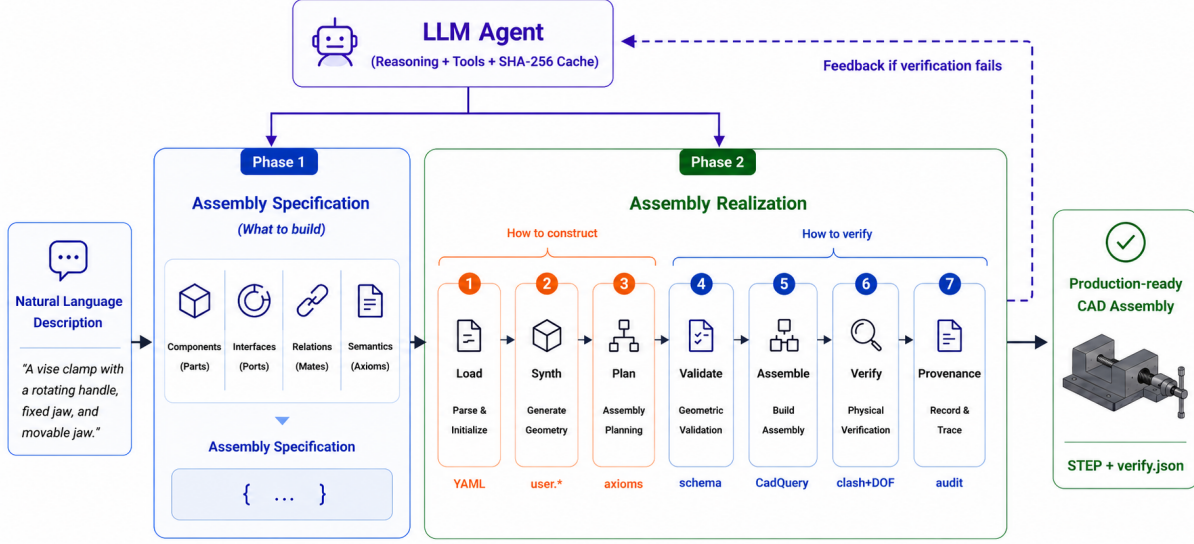


Figure 3: Overview of ASSEMBLYCAD. The framework first transforms natural-language descriptions into a structured Assembly Specification, and then realizes the specification through an execution pipeline for geometry construction and engineering verification. Verification failures are fed back to the LLM agent for specification refinement, while successful executions produce production-ready CAD assemblies together with verification reports.

Rather than treating assembly relationships as purely geometric connections, each mate in the assembly specification is associated with one or more engineering axioms that justify its existence and configuration. These axioms provide a shared semantic layer throughout the entire pipeline: they guide semantic decomposition, support component synthesis, and are propagated to the final verification report. Consequently, assembly decisions are grounded not only in geometric validity, but also in interpretable engineering principles.

3.3 Assembly Generation

Based on the assembly specification and engineering semantics, the production-ready CAD assembly generation task is formally defined as follows.

Definition 2 (Assembly Generation). *Given a natural-language description $d \in \Sigma^*$, the goal is to produce both an assembly specification $\mathcal{S}$ and its geometric realization*

$$G : \mathcal{S} \rightarrow \mathbb{R}^3,$$

such that:

1. Each part $p_i \in \mathcal{P}$ is instantiated as a valid B-Rep solid with positive volume.
2. Every mate $m_j \in \mathcal{M}$ is geometrically satisfiable under G .
3. The resulting assembly graph $(\mathcal{P}, \mathcal{M})$ is connected with a designated root component.
4. No unintended interference exists, i.e., for any $(p_i, p_j) \notin \mathcal{E}_{\text{expected}}$, the intersection volume between the corresponding solids is below a predefined threshold τ .

4 AssemCAD

4.1 System Overview

Building upon the problem formulation in Section 3, ASSEMBLYCAD decomposes production-ready CAD assembly generation into two complementary stages: **Assembly Specification** and **Assembly Realization**, as illustrated in Figure 3. The framework first converts natural-language descriptions into a structured Assembly Specification using an LLM agent, and subsequently realizes the specification through a deterministic realization pipeline. Verification failures are fed back to the LLM agent for iterative refinement, while successful executions produce production-ready CAD assemblies together with verification reports. Unlike conventional end-to-end CAD generation systems, ASSEMBLYCAD explicitly separates semantic reasoning from geometric realization, allowing each stage to be independently validated and refined through verification feedback.

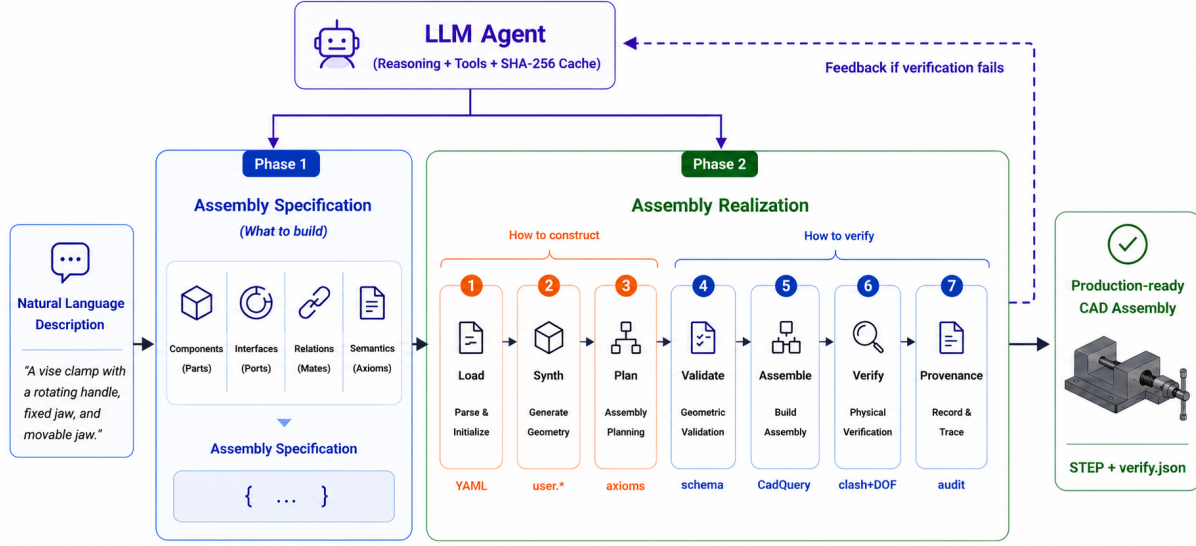


图3: assemcad系统概述。该框架首先将自然语言描述转换为结构化的装配规范，随后通过一个用于几何构建与工程验证的执行流水线来实现该规范；验证失败结果会反馈至LLM智能体以进行规范优化，而成功的执行过程则可生成可直接投入生产的CAD装配件及相应的验证报告。

与将装配关系视为纯粹的几何连接不同，装配规范中的每个配对对象均关联着一个或多个工程公理，这些公理为该配对对象的存在及其配置提供了依据。这些公理在整个流程中构建了一个共享的语义层：它们指导语义分解、支持组件合成，并被传递至最终的验证报告中。因此，装配决策不仅基于几何有效性，还基于可解释的工程原则。

3.3 装配生成

基于装配规范与工程语义，可直接用于生产的CAD装配生成任务可正式定义如下：

定义2（装配生成）：给定自然语言描述 $d \in \Sigma^*$ ，目标是同时生成装配规范 S 及其几何实现。

$$G : S \rightarrow \mathbb{R}^3,$$

到这样的程度：

1. 每个部分 $p_i \in P$ 均被实例化为一个具有正体积的有效B-Rep实体。
2. 每个伴侣 $m_j \in M$ 在图 G 下均是几何可满足的。
3. 生成的装配图 (P, M) 与指定的根组件相连。
4. 不存在任何非预期干扰，即对于任意 $(p_i, p_j) \notin \mathcal{E}_{\text{expected}}$ 下，相应实体之间的交集体积低于预定义阈值 τ 。

4 AssemCAD

4.1 系统概述

基于第3节中的问题建模，assemcad将生产级CAD装配生成过程分解为两个互补阶段：**装配规范**与**装配实现**，如图3所示。该框架首先利用LLM智能体将自然语言描述转换为结构化的装配规范，随后通过确定性实现流程执行该规范。验证失败会反馈至LLM智能体以进行迭代优化，而成功的执行则可生成生产级CAD装配件及相应的验证报告。与传统的端到端CAD生成系统不同，assemcad明确区分了语义推理与几何实现环节，使得每个阶段均可通过验证反馈独立进行验证与优化。

Algorithm 1 Structured Decomposition with Bounded Repair**Require:** NL description d , LLM client $\mathcal{L}$, max rounds K **Ensure:** Valid AssemblyDraft $\hat{S}$ or error

```

1:  $\hat{S} \leftarrow \mathcal{L}(d)$ 
2: for  $k = 1, \dots, K$  do
3:    $(valid, errors) \leftarrow \text{VALIDATEDRAFT}(\hat{S})$ 
4:   if  $valid$  then
5:     return  $\hat{S}$ 
6:    $\hat{S} \leftarrow \mathcal{L}(d, errors)$ 
7: raise DecompositionError

```

Table 2: Mate types with geometric semantics and configurable options.

Mate Type τ	Constraint	Options
face_to_face	Coplanar normals	u, v, d_n
coaxial	Shared axis	α, d
coaxial_face	Axis + contact	d
gear_mesh	Pitch tangency	m, r
press_fit	Interference fit	d
thread_engage	Thread engagement	d_t
snap_to_face	Surface snap	u, v, d_n

Assembly Specification. Given a natural-language description, the LLM agent generates a structured Assembly Specification that explicitly captures components, interfaces, assembly relationships, and engineering semantics, providing a semantic representation of the target assembly.

Assembly Realization. Given the Assembly Specification, the realization pipeline progressively constructs, assembles, and verifies the corresponding CAD assembly through seven sequential stages, ensuring geometric validity, physical consistency, and traceable execution.

4.2 Phase I: What to Build

The first phase determines *what to build*. Given a natural-language description, ASSEMBCAD performs semantic decomposition and constructs an *Assembly Specification*, which serves as the interface between language understanding and the downstream deterministic pipeline. Rather than directly generating geometry, the system first identifies components, interfaces, assembly relationships, and engineering semantics, thereby separating semantic intent from geometric realization.

Given a natural-language description, the LLM decomposes the design intent into an AssemblyDraft containing component factories, typed ports, mate relationships, and engineering semantics. Since the generated specification may violate structural constraints, a bounded repair loop is employed to validate and iteratively refine the draft. Algorithm 1 summarizes the decomposition procedure.

To explicitly model assembly interfaces, spatial relationships between parts are mediated through *ports*—typed geometric attachment points declared by each factory. Unlike conventional CadQuery assemblies, where rigid transformations are directly specified between parts, ASSEMBCAD represents connectivity through typed ports and mates. This interface-centric abstraction decouples assembly semantics from geometric realization and enables reusable attachment patterns across heterogeneous components. In our implementation, we define 12 port types and 7 mate types, summarized in Table 2.

Definition 3 (Port-Mate Compatibility). *A mate of type $\tau \in \mathcal{T}$ is said to be compatible with a port pair (π_b, π_c) if and only if $(t_b, t_c) \in C_\tau$, where $C_\tau \subseteq \mathcal{T}_{\text{port}} \times \mathcal{T}_{\text{port}}$ denotes the compatibility relation associated with mate type τ .*

Each mate specifies both geometric semantics and configurable parameters. For example, the `face_to_face` mate supports in-plane offsets (u, v) and normal displacement d_n , allowing multiple components to reference the same base interface without geometric coincidence. This mechanism differs fundamentally from conventional rigid transformations and provides a structured foundation for subsequent construction and geometry-aware validation.

Beyond geometric semantics, each mate in the Assembly Specification is further grounded by referencing one or more engineering axioms from the axiom set $\mathcal{A}_{\text{axioms}}$ introduced in Section 3. During decomposition, the LLM is required to explicitly justify every mate by selecting appropriate axioms. For instance, a `coaxial` mate connecting a shaft to a bearing must reference axiom C-02 (clearance-fit constraint), while a `gear_mesh` mate must reference axiom G-02 (module and pressure-angle compatibility). This axiom grounding serves three purposes: (1) it constrains the LLM’s assembly reasoning by requiring engineering justification rather than purely geometric plausibility; (2) it makes the resulting specification interpretable, as each assembly relation carries an explicit engineering rationale traceable to the axiom set; and (3) it enables downstream verification to check not only geometric consistency but also engineering-rule compliance. Once a valid and axiom-grounded Assembly Specification is obtained, the downstream pipeline is responsible for constructing and verifying the corresponding CAD assembly.

算法1: 有界修复的结构分解**Require:** NL description d , LLM client $\mathcal{L}$, max rounds K **Ensure:** Valid AssemblyDraft $\hat{S}$ or error

```

1:  $\hat{S} \leftarrow \mathcal{L}(d)$ 
2: for  $k = 1, \dots, K$  do
3:    $(valid, errors) \leftarrow \text{VALIDATEDRAFT}(\hat{S})$ 
4:   若  $valid$  则
5:     返回  $\hat{S}$ 
6:    $\hat{S} \leftarrow \mathcal{L}(d, errors)$ 
7: raise DecompositionError

```

表2: 具有几何语义及可配置选项的Mate类型。

Mate 类型 τ	约束	选项
面对面同轴	共面法线: 共享轴	u, v, d_n
同轴面齿轮-网	轴线+接触、轴向	α, d, d
格-压配-螺纹啮	公比切线、过盈	m, r
合	配合、螺纹啮	d_t
<code>snap_to_face</code>	合、表面自锁	u, v, d_n

装配规范: 给定自然语言描述, LLM智能体会生成一个结构化的装配规范, 该规范可明确捕获组件、接口、装配关系及工程语义, 从而为目标装配提供语义表示。

装配实现: 基于装配规范, 实现流程通过七个连续阶段, 逐步构建、装配并验证相应的CAD装配模型, 确保其几何有效性、物理一致性及可追溯执行性。

4.2 第一阶段: 需构建的内容

第一阶段确定需构建的内容。给定自然语言描述时, AssemCAD会进行语义分解, 并构建一个装配规范——该规范充当了语言理解与下游确定性处理流程之间的接口。系统并非直接生成几何模型, 而是先识别出组件、接口、装配关系及工程语义, 从而将语义意图与几何实现相分离。

给定自然语言描述, 大语言模型会将设计意图分解为一个AssemblyDraft, 其中包含组件工厂、类型化端口、匹配关系及工程语义。由于生成的规范可能违反结构约束, 系统采用有限修复循环来验证并迭代优化该草案。算法1总结了该分解流程。

为明确建模装配接口, 零件之间的空间关系是通过端口——即由各工厂声明的带类型标注的几何附着点——来介导的。与传统的CadQuery装配方式不同(后者在零件间直接指定刚性变换), AssemCAD通过带类型标注的端口和配合来表示连接关系。这种以接口为中心的抽象方法将装配语义与几何实现解耦, 从而支持跨异构组件的可复用附着模式。在我们的实现中, 我们定义了12种端口类型和7种配合类型, 汇总于表2中。

定义3 (端口匹配兼容性): 若类型 $\tau \in T$ 的配对对象与端口对 (π_b, π_c) 相容, 当且仅当 $(t_b, t_c) \in C_\tau$, where $C_\tau = \mathcal{T}_{\text{port}} \times \mathcal{T}_{\text{port}}$ 表示与配对类型 τ 相关的兼容关系。

每个配对对象均同时定义几何语义与可配置参数。例如, `face_to_face` 配对对象支持面内偏移量 (u, v) 及法向位移 d_n , 使得多个组件能够引用同一基础接口而无需满足几何重合条件。该机制与传统刚性变换存在根本差异, 为后续构建过程及几何感知验证提供了结构化基础。

除几何语义外, 装配规范中的每个配对关系还需通过引用第3节中引入的公理集 $\mathcal{A}_{\text{axioms}}$ 中的一个或多个工程公理来进一步奠定基础。在分解过程中, 大语言模型需通过选择适当的公理, 来明确论证每一个配对关系。例如, 将轴与轴承连接的同轴配对必须引用公理C-02(间隙配合约束), 而齿轮啮合配对则必须引用公理G-02(模数与压力角兼容性)。这种公理奠基具有三个目的: (1) 通过要求大语言模型进行工程层面的论证而非仅基于几何合理性, 从而约束其装配推理过程; (2) 使最终生成的规范可被解读——因为每种装配关系都附带了可追溯至公理集的明确工程依据; (3) 为下游验证流程提供支持, 使其不仅能检查几何一致性, 还能验证是否符合工程规则。一旦获得有效且经公理奠基的装配规范, 下游流程便负责构建并验证相应的CAD装配模型。

Algorithm 2 Factory Synthesis with Three-Layer Validation

Require: Description δ , expected ports Π_{exp} , parameters θ_0 , LLM client $\mathcal{L}$, max rounds K

Ensure: Validated factory module $\mathcal{M}$ or error

```
1:  $k_{\text{cache}} \leftarrow \text{SHA256}(\delta \parallel \Pi_{\text{exp}} \parallel \theta_0)$ 
2: if  $k_{\text{cache}} \in \text{Cache}$  then
3:   return  $\text{Cache}[k_{\text{cache}}]$ 
4:  $\text{code} \leftarrow \mathcal{L}.\text{Chat}(\text{SynthPrompt}(\delta, \Pi_{\text{exp}}, \theta_0))$ 
5: for  $k = 1, \dots, K$  do
6:    $\text{report} \leftarrow \text{Sandbox}(\text{code}, \theta_0)$ 
7:   if  $\text{report.passed}$  then
8:      $\text{Cache}[k_{\text{cache}}] \leftarrow \text{code}$ 
9:   return  $\text{code}$ 
10:  $\text{code} \leftarrow \mathcal{L}.\text{Chat}(\text{RepairPrompt}(\text{code}, \text{report}))$ 
11: raise  $\text{SynthesisError}(\text{report})$ 
```

4.3 Phase II: How to Construct

The second phase determines *how to construct individual components*. Given an Assembly Specification, ASSEMBCAD transforms symbolic parts into executable geometry. This phase is built upon a port- and mate-based CAD assembly library that provides both a registry of reusable parametric component factories and an on-demand synthesis mechanism for open-world geometries.

Assembly Library and Factory Registry. The assembly library extends CadQuery with a typed port system and deterministic mate semantics (detailed in Appendix B). At its core, the library maintains a registry of 13 built-in parametric factories spanning common mechanical components including structural primitives (`rect_plate`, `l_bracket`, `u_channel`, `housing_block`), rotational components (`stepped_shaft`, `coupling_hub`, `boss`), and interface elements (`bolt_circle_flange`), etc. Additionally, the library provides adapter modules that wrap third-party parametric libraries—`cq_warehouse` for standardized fasteners (bolts, nuts, washers) and bearings, and `cq_gears` for involute spur gears, augmenting each component with typed port annotations. Every factory in the registry produces not only valid B-Rep geometry but also a set of typed ports that are compatible with the port-mate abstraction defined in Phase I. This co-location of geometry construction and port declaration prevents the geometry-metadata drift that commonly arises when ports are annotated separately from their geometric features.

On-Demand Component Synthesis. For components beyond the built-in registry, ASSEMBCAD supports open-world synthesis by generating previously unseen factories on demand via the LLM. Each component is associated with a parameterized generator. During execution, ASSEMBCAD first attempts to retrieve an existing implementation from the factory registry. If no matching implementation exists, the component is synthesized on demand by the LLM. The generated CadQuery module exposes both geometric parameters and the ports declared in Phase I, thereby ensuring compatibility with the port-mate abstraction.

To avoid redundant synthesis, each request is identified by a content hash computed from the component description, expected ports, and parameter specification. Existing implementations are directly reused, whereas unseen requests trigger a synthesis procedure driven by the LLM.

Unlike template-based CAD generation methods, executable code alone is insufficient. Generated components must simultaneously satisfy geometric validity and interface correctness. Therefore, ASSEMBCAD employs an iterative generation-repair loop equipped with a three-layer validation mechanism. Validation failures are converted into repair feedback and fed back to the LLM until a valid implementation is obtained or the retry budget is exhausted. Algorithm 2 summarizes the complete synthesis procedure.

The first two validation layers ensure successful execution and geometric validity. The final layer verifies the consistency between declared ports and the generated geometry, ensuring that interfaces are supported by explicit geometric evidence rather than symbolic annotations alone.

Definition 4 (Port-Geometry Consistency). A port

$$\pi = (\text{name}, t, \mathbf{o}, \hat{\mathbf{n}}, \phi)$$

on solid S satisfies port-geometry consistency if and only if the type-specific geometric evidence holds:

$$\text{Consistent}(\pi, S) \triangleq \begin{cases} \text{CavityAt}(\mathbf{o}, \hat{\mathbf{n}}, \phi_d, S), & t = \text{bore}, \\ \text{FaceAt}(\mathbf{o}, \hat{\mathbf{n}}, S), & t = \text{flat_face}, \\ \text{MaterialAlong}(\mathbf{o}, \hat{\mathbf{n}}, \phi_L, S), & t = \text{shaft_seat}. \end{cases} \quad (1)$$

算法2: 三层验证下的工厂合成

Require: Description δ , expected ports Π_{exp} , parameters θ_0 , LLM client $\mathcal{L}$, max rounds K

Ensure: Validated factory module $\mathcal{M}$ or error

```
1:  $k_{\text{cache}} \leftarrow \text{SHA256}(\delta \parallel \Pi_{\text{exp}} \parallel \theta_0)$ 
2: if  $k_{\text{cache}} \in \text{Cache}$  then
3:   return  $\text{Cache}[k_{\text{cache}}]$ 
4:  $\text{code} \leftarrow \mathcal{L}.\text{Chat}(\text{SynthPrompt}(\delta, \Pi_{\text{exp}}, \theta_0))$ 
5: for  $k = 1, \dots, K$  do
6:    $\text{report} \leftarrow \text{Sandbox}(\text{code}, \theta_0)$ 
7:   if  $\text{report.passed}$  then
8:      $\text{Cache}[k_{\text{cache}}] \leftarrow \text{code}$ 
9:   return  $\text{code}$ 
10:  $\text{code} \leftarrow \mathcal{L}.\text{Chat}(\text{RepairPrompt}(\text{code}, \text{report}))$ 
11: raise  $\text{SynthesisError}(\text{report})$ 
```

4.3 第二阶段：如何构建

第二阶段确定如何构建各个组件。给定装配规范AS SEMCAD，可将符号部件转换为可执行几何模型。该阶段基于一个基于端口和配对的CAD装配库构建，该库不仅提供了可复用参数化组件工厂的注册表，还提供了针对开放世界几何模型的按需综合机制。

装配库与工厂注册表。该装配库通过引入类型化端口系统和确定性配对语义（详见附录B）扩展了Cad-Query功能。其核心在于维护一个包含13个内置参数化工厂的注册表，这些工厂涵盖常见的机械部件，包括结构原语（rect_plate、l_bracket、u_channel、housing_block）、旋转部件（stepped_shaft、coupling_hub、boss）以及接口元件（bolt_circle_flange）等。此外，该库还提供了适配器模块，可封装第三方参数化库——如用于标准化紧固件（螺栓、螺母、垫圈）和轴承的cq_warehouse，以及用于渐开线直齿圆柱齿轮的cq_gears，从而为每个组件添加类型化端口注释。注册表中的每个工厂不仅能生成有效的B-Rep几何体，还能生成一组与第一阶段定义的端口-配对抽象相兼容的类型化端口。这种几何构造与端口声明的同处一地设计，有效避免了当端口与其几何特征分别注释时常见的几何-元数据脱节问题。

按需组件合成。对于超出内置注册表范围的组件，AssemCAD支持开放世界合成，即通过大语言模型（LLM）按需生成此前未见过的工厂。每个组件均关联一个参数化生成器。在执行过程中，AssemCAD首先尝试从工厂注册表中检索现有实现；若未找到匹配的实现，则由LLM按需合成该组件。生成的CadQuery模块会公开几何参数及第一阶段声明的端口，从而确保与端口匹配抽象层的兼容性。

为避免冗余合成，每个请求均通过由组件描述、预期端口及参数规范计算得出的内容哈希值进行标识。现有实现会直接复用该哈希值，而未见过的请求则会触发由LLM驱动的合成流程。

与基于模板的CAD生成方法不同，仅凭可执行代码是不够的。生成的组件必须同时满足几何有效性与接口正确性要求。因此，AssemCAD采用了包含三层验证机制的迭代生成-修复循环：验证失败会被转换为修复反馈，并反馈至大语言模型（LLM），直至获得有效的实现方案或耗尽重试预算。算法2总结了完整的合成流程。

前两个验证层可确保执行成功与几何有效性；最后一层则验证声明的端口与生成的几何结构之间的一致性，从而确保接口由明确的几何证据而非仅依赖符号注释来支持。

定义4（端口几何一致性）。端口

$$\pi = (\text{name}, t, \mathbf{o}, \hat{\mathbf{n}}, \phi)$$

在固体 S 上，端口几何一致性成立当且仅当类型特异性几何证据成立：

$$\text{Consistent}(\pi, S) \triangleq \begin{cases} \text{CavityAt}(\mathbf{o}, \hat{\mathbf{n}}, \phi_d, S), & t = \text{bore}, \\ \text{FaceAt}(\mathbf{o}, \hat{\mathbf{n}}, S), & t = \text{flat_face}, \\ \text{MaterialAlong}(\mathbf{o}, \hat{\mathbf{n}}, \phi_L, S), & t = \text{shaft_seat}. \end{cases} \quad (1)$$

Algorithm 3 Assembly Construction and Verification

Require: Validated tree $\mathcal{T} = (\mathcal{P}, \mathcal{M})$, clash threshold τ

Ensure: Assembly $\mathcal{A}$, verification report $\mathcal{V}$

```
1:  $\mathcal{A} \leftarrow \text{ASSEMBLYEXT}()$ 
2: for  $p_i \in \mathcal{P}$  do
3:    $w_i \leftarrow f_i.\text{BUILD}(\theta_i)$ 
4:    $\mathcal{A}.\text{ADD}(w_i, \text{id}_i)$ 
5: for  $m_j = (\tau_j, b_j, c_j, \omega_j) \in \mathcal{M}$  do
6:    $T_j \leftarrow \text{COMPUTETRANSFORM}(\tau_j, \mathcal{A}.\text{PORT}(b_j), \mathcal{A}.\text{PORT}(c_j), \omega_j)$ 
7:    $\mathcal{A}.\text{LOCATE}(c_j.\text{ref}, T_j)$ 
   // Verification
8: for  $(p_i, p_j) \in \binom{\mathcal{P}}{2}$  do
9:    $v_{ij} \leftarrow \text{Vol}(\text{BREPCOMMON}(\mathcal{A}[p_i], \mathcal{A}[p_j]))$ 
10:  if  $v_{ij} > \tau$  then
11:     $\text{classify via Eq. (3)}$ 
12:  $\mathcal{V}_{\text{dof}} \leftarrow \text{BFS}(\mathcal{A}, \mathcal{M}, p_1)$ 
13: return  $(\mathcal{A}, \mathcal{V})$ 
```

where CavityAt requires sample points at radial offset $\phi_d/4$ to be classified as OUT by BRepClass3d_SolidClassifier, FaceAt requires a coincident face with collinear normal (within 10^{-3} rad), and MaterialAlong requires IN classification at $\pm\phi_L/2$ along $\hat{\mathbf{n}}$.

This definition elevates ports from symbolic annotations to verifiable geometric interfaces. For example, a bore port is valid only when a cylindrical cavity exists at the declared location, while a shaft-seat port requires material support along the specified axis. Such consistency guarantees that assembly interfaces are grounded in geometry rather than merely declared in the specification.

Consequently, generator synthesis and interface validation are tightly coupled. Every admitted component is guaranteed not only to produce valid geometry, but also to expose physically meaningful interfaces that are compatible with downstream assembly operations.

4.4 Phase III: How to Verify

The third phase determines *whether the generated assembly is production-ready*. Given validated component factories and an Assembly Specification, ASSEMBLY constructs the final assembly and subsequently performs geometry-aware verification to ensure physical consistency. This phase is realized through the assembly library’s deterministic assembly engine, which operationalizes the port-mate abstraction through closed-form mate transforms and multi-layer verification.

Deterministic Mate Transform. Unlike conventional CAD assembly approaches that rely on iterative constraint solvers or manually specified rigid transformations, the assembly library computes each mate as a closed-form rigid-body transform. Given a base port π_b (already placed in world coordinates) and an incoming port π_c (in the incoming part’s local frame), the transform that places the incoming part is:

$$T = L_b \cdot R_{\text{flip}} \cdot R_\alpha \cdot L_c^{-1}, \quad (2)$$

where $L_b, L_c \in SE(3)$ are the respective port frames, R_{flip} is a 180° rotation about the local x -axis (making z -axes anti-parallel for face-to-face contact), and R_α is an optional user-specified rotation about the shared z -axis. Each of the seven mate types (Table 2) extends this base transform with type-specific semantics: for example, `gear_mesh` validates matching module and pressure angle before computing the center distance, while `press_fit` verifies the interference condition before delegating to coaxial alignment. This single matrix multiplication produces byte-identical results across runs, satisfying Proposition 2, and eliminates the non-determinism inherent in iterative solvers.

Assembly Construction and Verification. Algorithm 3 summarizes the complete assembly and verification process. Starting from instantiated parts, the system iteratively applies mate constraints through port alignment using Eq. (2) and computes relative transformations between components. Before each mate is applied, the library checks port-type compatibility via the predefined compatibility matrix (Definition 3), catching category errors (e.g., mating a fastener port to a gear mesh port) before geometric computation. Once the assembly is constructed, pairwise interference analysis and connectivity verification are performed to generate the final report. The output consists of a STEP model together with a verification report.

算法3组件构建与验证

要求：经过验证的树 $T=(P, M)$ ，冲突阈值 τ

确保：组装件 A 、验证报告 V

```
1:  $A \leftarrow \text{assemblyext}()$ 
2: 对于  $p_i \in P$ ，执行以下操作：
3:    $w_i \leftarrow f_i.\text{build}(\mathcal{O}_i)$ 
4:    $A.\text{ADD}(w_i, \text{id}_i)$ 
5: 对于  $m_j = (\tau_j, b_j, c_j, \omega_{f_j}) \in M$ ，执行以下操作：
6:    $T_j \leftarrow \text{计算变换}(\tau_j, A.\text{PORT}(B_j), A.\text{PORT}(c_j), \omega_j)$ 
7:    $A.\text{定位}(c_j.\text{ref}, T_j)$ 
   // 验证
8: 对于  $(p_i, p_j) \in (P_2)$ ，执行以下操作：
9:    $v_{ij} \leftarrow \text{Vol}(\text{brepcommon}(A[p_i], A[p_j]))$ 
10: 若  $v_{ij} > \tau$ ，则
11:   | 通过等式 (3) 进行分类
12:  $V_{\text{dof}} \leftarrow \text{BFS}(A, M, p_l)$ 
13: 返回  $(A, V)$ 
```

其中，CavityAt要求径向偏移量 $\phi_d/4$ 处的采样点被BRepClass3d_SolidClassifier分类为“OUT”；FaceAt要求存在一个与法线共线的重合面（误差在 10^{-3} 弧度以内）；而MaterialAlong则要求沿 $\hat{\mathbf{n}}$ 方向、位于 $\pm \phi_L/2$ 处的区域被分类为“IN”。

这一定义将端口从象征性注释提升为可验证的几何接口。例如，孔端口仅在声明位置存在圆柱形空腔时才有效，而轴座端口则需要沿指定轴线提供材料支撑。这种一致性确保了装配接口是基于几何结构而非仅仅在规范中声明的。

因此，生成器合成与接口验证紧密耦合。每个被接受的组件不仅保证能生成有效的几何结构，还保证能暴露具有物理意义的接口，这些接口与下游装配操作兼容。

4.4 第三阶段：如何验证

第三阶段用于确定生成的装配体是否已达到生产就绪状态。在已有经过验证的组件工厂和装配规范的前提下，AssemCAD会构建最终装配体，并随后执行几何感知验证以确保物理一致性。该阶段通过装配库中的确定性配对引擎来实现，该引擎通过闭式配对变换和多层验证来实现端口匹配抽象。

确定性配对变换。与依赖于迭代约束求解器或手动指定刚体变换的传统CAD装配方法不同，该装配库将每个配对计算为闭式刚体变换。给定基端口 π_b （已置于世界坐标系中）和输入端口 π_c （位于输入部件的局部坐标系内），用于定位输入部件的变换为：

$$T = L_b \cdot R_{\text{flip}} \cdot R_\alpha \cdot L_c^{-1}, \quad (2)$$

其中 $L_b, L_c \in SE(3)$ 分别为各自的端口坐标系， R_{flip} 表示绕局部 x 轴进行 180° 旋转（使 z 轴在面对面接触时呈反平行关系），而 R_α 则是可由用户指定的绕共享 z 轴的旋转操作。七种配合类型（表2）均通过添加特定于该类型的语义来扩展此基础变换：例如，`gear_mesh` 会在计算中心距前验证匹配模块与压力角是否符合；而 `press_fit` 则会在委托执行同轴对齐之前，先验证干涉条件。这一单一矩阵乘法运算可在不同运行中产生完全一致的字节结果，满足命题2，并消除了迭代求解器固有的非确定性问题。

装配构建与验证。算法3总结了完整的装配与验证流程。系统从实例化部件开始，通过使用等式 (2) 进行端口对齐，迭代地应用配合约束，并计算各组件之间的相对变换关系。在每次应用配合前，库会借助预定义的兼容性矩阵（定义3）检查端口类型兼容性，从而在几何计算前捕获类别错误（例如：将紧固件端口与齿轮啮合端口进行配合）。一旦完成装配构建，系统将执行成对干涉分析和连通性验证，以生成最终报告。输出结果包括一个STEP模型及一份验证报告。

During assembly, boolean operations may silently produce disconnected compounds when solids only touch at boundaries. To prevent such failures, ASSEMCAD employs a robust union operator, `SAFEUNION`, which explicitly preserves connectivity and raises errors whenever a valid fusion cannot be established.

Proposition 1 (`SAFEUNION` Correctness). *Let B and P be two `CadQuery` solids with*

$$|\text{Solids}(B)| = |\text{Solids}(P)| = 1.$$

Then `SAFEUNION`(B, P, ε) either returns a solid R satisfying

$$|\text{Solids}(R)| = 1,$$

or raises an error. In particular, it never silently returns a disconnected compound.

Proof sketch. `SAFEUNION` checks three cases sequentially. First, if $\text{Vol}(B \cap P) > 0$, standard union preserves connectivity. Second, if B and P share a common face, glue-based fusion is applied. Otherwise, one component is extended by a small tolerance ε along the shortest-gap direction to create volumetric overlap before union. Finally, the post-condition $|\text{Solids}(R)| = 1$ is explicitly asserted; any violation raises an error instead of propagating a disconnected compound.

Unlike conventional collision detection, interference analysis must be aware of assembly semantics. Certain mate types inherently require geometric overlap and therefore should not be classified as collisions. Specifically,

$$\mathcal{T}_{\text{contact}} = \{\text{gear_mesh}, \text{press_fit}, \text{thread_engage}\}$$

denotes mate types with intentional geometric interference. Consequently, clash classification is mate-type-aware:

$$\text{Class}(p_i, p_j) = \begin{cases} \text{expected}, & \exists m \in \mathcal{M} : m \text{ connects } (p_i, p_j) \wedge \tau_m \in \mathcal{T}_{\text{contact}}, \\ \text{clash}, & \text{otherwise.} \end{cases} \quad (3)$$

Therefore, geometric overlap does not necessarily indicate invalidity. Instead, interference is interpreted according to the semantics of the corresponding mate. This distinction enables ASSEMCAD to correctly handle assemblies involving gears, press fits, and threaded engagements, where contact and penetration are physically necessary.

Besides pairwise interference analysis, the verification stage also checks graph connectivity and degrees of freedom. Connectivity is verified by traversing the assembly graph, while remaining degrees of freedom are computed from the induced mate constraints. The resulting verification report records clashes, connectivity status, and mobility information, and is exported together with the final STEP model.

Proposition 2 (Determinism). *For any fixed input description d and LLM response cache $\mathcal{C}$, the pipeline output $G(\mathcal{S})$ is uniquely determined. Formally, if*

$$\mathcal{C}(q) = r$$

for every query encountered during execution, then repeated invocations produce byte-identical STEP files and verification reports.

Proof sketch. The result follows directly from the content-addressed architecture of ASSEMCAD. LLM responses are cached by SHA256 hashes, generator synthesis is keyed by specification hashes, and all downstream computation is purely functional. Since assembly construction and verification contain no stochastic operations, identical inputs necessarily produce identical geometric realizations and identical reports.

Axiom Traceability. Beyond geometric verification, the axiom references attached to each mate during specification (Phase I) are propagated through assembly construction to the final verification report. This creates an auditable chain from the output STEP file, through the assembly tree, back to the engineering axiom and ultimately to its textbook provenance. For example, if a clash is detected between two gears, the report not only identifies the interference volume but also references axiom G-02 (module compatibility), enabling engineers to trace whether the failure stems from a specification error or a geometric inconsistency. This axiom traceability distinguishes ASSEMCAD from conventional CAD pipelines, where verification results lack engineering semantics and are purely geometric in nature.

Consequently, ASSEMCAD transforms symbolic assembly specifications into reproducible CAD artifacts with explicit physical and engineering guarantees. The resulting STEP model and verification report together provide a production-ready representation suitable for downstream engineering and manufacturing workflows.

在装配过程中，当实体仅在边界处接触时，布尔运算可能会静默地生成不连通的复合体。为防止此类故障，AssemCAD采用了稳健的“safeunion”并集运算符——该运算符能明确地保持连通性，并在无法建立有效的融合时抛出错误。

命题1（safeunion 正确性）。设 B 和 P 为两个 CadQuery 实体，且

$$|\text{固体}(B)| = |\text{固体}(P)| = 1。$$

随后， $\text{safeunion}(B, P, \emptyset)$ 会返回一个满足条件的实数集 R

$$|\text{固体}(R)| = 1，$$

或会抛出错误；具体而言，它绝不会静默地返回一个已断开连接的复合对象。

证明概要：‘safeunion’函数依次检查三种情况：首先，若 $\text{Vol}(B \cap P) > 0$ ，标准并集操作可保持连通性；其次，若 B 与 P 共享公共面，则采用基于粘合的融合方法；否则，沿最短间隙方向通过引入微小容差 ϵ 对其中一个组件进行扩展，以在并集操作前创建体积重叠区域；最后，显式断言后置条件 $\text{Vol}(\text{Solids}(R)) = 1$ ；任何违反该条件的情况都会触发错误提示，而非产生不连通的复合体。

与传统的碰撞检测不同，干涉分析必须充分考虑装配语义。某些配对类型在本质上需要几何重叠，因此不应被归类为碰撞。具体而言：

$$\mathcal{T}_{\text{contact}} = \{\text{gear_mesh}, \text{press_fit}, \text{thread_engage}\}$$

表示具有有意几何干涉的配对类型；因此，冲突分类是配对类型感知的：

$$\text{Class}(p_i, p_j) = \begin{cases} \text{expected}, & \exists m \in \mathcal{M} \text{ connects } (p_i, p_j) \text{ 且 } m \in \mathcal{T}_{\text{contact}}, \text{ 否} \\ \text{clash}, & \text{则。} \end{cases} \quad (3)$$

因此，几何重叠并不一定意味着装配无效。相反，干涉关系需根据相应配对部件的语义来解读。这一区分使得 AssemCAD 能够正确处理涉及齿轮、过盈配合及螺纹接合的装配场景——在这些场景中，接触与穿透在物理上是必要的。

除成对干扰分析外，验证阶段还会检查图的连通性与自由度。连通性通过遍历装配图来验证；而其余自由度则根据诱导的配对约束计算得出。生成的验证报告会记录碰撞冲突、连通性状态及运动自由度信息，并随最终 STEP 模型一同导出。

命题2（确定性）：对于任意固定的输入描述 S 和 LLM 响应缓存 C ，流水线输出 $G(S)$ 均可唯一确定。形式化地，若

$$\mathcal{C}(q) = r$$

对于执行过程中遇到的每个查询，重复调用将生成完全相同的字节格式 STEP 文件及验证报告。

证明概要。该结果可直接由 assemcad 的内容寻址架构得出：LLM 响应通过 SHA256 哈希值进行缓存，生成器合成基于规范哈希值进行标识，且所有下游计算均采用纯函数式范式。由于汇编构建与验证过程不包含任何随机操作，相同的输入必然会产生完全一致的几何实现及完全一致的报告结果。

公理可追溯性。除几何验证外，规范阶段（第一阶段）中与每个配合件关联的公理引用，会通过装配构建过程传递至最终验证报告。这形成了从输出的 STEP 文件、经装配树、回溯至工程公理，最终追溯至其教材来源的可审计链路。例如，若检测到两个齿轮之间存在碰撞，则该报告不仅能识别干涉体积，还能引用公理 G-02（模数兼容性），使工程师能够追溯故障是源于规范错误还是几何不一致。这种公理可追溯性正是 AssemCAD 与传统 CAD 流程的区别所在——后者生成的验证结果缺乏工程语义，本质上仅为几何数据。

因此，Consemcad 能够将符号化装配规范转换为可复现的 CAD 模型，该模型具备明确的物理与工程保障。最终生成的 STEP 模型及验证报告共同构成了一套可直接投入生产的表示方案，适用于下游的工程与制造流程。

Text-to-CAD Assembly Generation					Image-Text-to-CAD Assembly Generation				
Method	SR↑	APR↑	VLM Judge		Method	SR↑	APR↑	VLM Judge	
			Assembly↑	Overall↑				Assembly↑	Overall↑
Cadrille [15]	65.00	—	—	—	✦ Gemini 3.5 Flash	30.83	0.00	3.21	5.41
✦ Qwen3.7-Max	78.33	39.17	2.88	4.53	✦ GLM-5.2	76.67	40.83	2.65	4.11
✦ GPT-5.5	79.17	66.67	2.96	4.73	✦ GPT-5.5	79.17	64.17	3.17	5.20
✦ Claude-4.8	85.00	0.83	2.70	4.37	✦ Claude Opus 4.8	65.83	0.00	2.92	4.74
AssemCAD (Ours)	89.17	87.50	3.38	4.53	AssemCAD (Ours)	84.17	82.50	3.48	4.77

Table 3: Comparison on ASSEMBENCH under two benchmark settings. SR denotes **Success Rate**, APR denotes the proposed **Assembly Preservation Rate**, and the last two columns are evaluated using the proposed **VLM-as-a-Judge**. Higher is better for all metrics.

5 Experiments

5.1 Benchmark Construction

Despite recent advances in natural-language CAD generation, there is currently no standardized evaluation protocol for production-ready CAD assembly generation. Existing CAD resources, including DeepCAD, Text2CAD, and Fusion360 Gallery [3, 13, 4, 23], are primarily designed for single-part modeling or data collection, and therefore do not evaluate assembly-level reasoning, inter-part relationships, or engineering correctness. To address this gap, we introduce ASSEMBENCH, a benchmark built upon the Fusion360 Gallery assembly corpus. Instead of collecting a new dataset, ASSEMBENCH establishes a standardized evaluation protocol by curating mechanically meaningful assemblies and pairing them with unified natural-language assembly specifications generated through a semi-automatic annotation pipeline (Appendix F). The benchmark is constructed according to three principles: (1) *engineering realism*, selecting functional mechanical assemblies with explicit engineering semantics; (2) *structural complexity*, requiring multiple interacting components and non-trivial assembly relationships; and (3) *parametric constructibility*, favoring assemblies that can be faithfully reconstructed using programmatic CAD operations. As summarized in Table 9, ASSEMBENCH contains 120 assembly instances spanning diverse mechanical structures and varying levels of assembly complexity. It is specifically designed to evaluate the task formulation proposed in this work, requiring models to jointly synthesize components, recover assembly structures, and satisfy engineering constraints. Detailed benchmark construction, statistics, and representative examples are provided in Appendix F.

5.2 Evaluation Protocol

We evaluate all methods under the two benchmark settings provided by ASSEMBENCH: **Text-to-CAD Assembly Generation** and **Image-Text-to-CAD Assembly Generation**. Since production-ready CAD assembly generation requires both executable geometry and correct assembly semantics, we evaluate each method using three complementary metrics: **Success Rate (SR)**, **Assembly Preservation Rate (APR)**, and **VLM-based Engineering Evaluation**.

Chamfer Distance. Although Chamfer Distance (CD) is widely adopted in prior 3D generation work, we deliberately exclude it from our primary evaluation. As we prove in Appendix H, CD is fundamentally unreliable for assembly evaluation: it lacks both *assembly structure identifiability* (Proposition 3) and *assembly ranking consistency* (Theorem 1). In particular, a structurally correct assembly can receive a worse CD score than one missing functionally critical components (Eq. 5). We therefore adopt assembly-aware metrics that directly evaluate structural correctness and engineering quality.

Success Rate (SR). SR measures the percentage of benchmark instances that successfully generate executable CAD artifacts without runtime errors. **Assembly Preservation Rate (APR).** APR measures the percentage of executable CAD programs that preserve the intended assembly structure. Assembly preservation is evaluated using an LLM-based code judge, where predictions with scores of at least 4 (out of 5) are regarded as successful. **VLM-based Engineering Evaluation.** Beyond executability and assembly preservation, we evaluate engineering quality using a VLM-as-a-Judge. Given the input specification, the reference rendering, and the generated rendering, the VLM reports two scores: **Assembly**, measuring structural correctness, and **Overall**, measuring overall engineering quality.

5.3 Main Results

Table 3 summarizes the quantitative results on ASSEMBENCH under both text-only and image-only assembly generation settings. We compare ASSEMCAD with representative CAD generation methods and frontier foundation

文本到CAD装配体生成					图像-文本至CAD装配体生成				
方法	SR↑	APR↑	VLM 法官		方法	SR↑	APR↑	VLM 法官	
			组装 / 总体 ↑					组装 / 总体 ↑	
Cadrille [15]	65.00	—	—	—	 Gemini 3.5 Flash	30.83	0.00	3.21	5.41
 Qwen3.7-Max	78.33	39.17	2.88	4.53	 GLM-5.2	76.67	40.83	2.65	4.11
 GPT-5.5	79.17	66.67	2.96	4.73	 GPT-5.5	79.17	64.17	3.17	5.20
 Claude-4.8	85.00	0.83	2.70	4.37	 克劳德作品第4.8号	65.83	0.00	2.92	4.74
AssemCAD (我们开发的版本)	89.17	87.50	3.38	4.53	AssemCAD (我们开发的版本)	84.17	82.50	3.48	4.77

表3：在两种基准测试设置下对assembench的比较结果。SR表示**成功率**，APR表示所提出的**装配保真率**，最后两列数据采用所提出的**VLM -as-aJudge**方法进行评估。所有指标值越高越好。

5 实验

5.1 基准测试构建

尽管近年来自然语言CAD生成技术取得了进展，但目前仍缺乏适用于生产级CAD装配生成的标准化评估方案。现有的CAD资源（包括DeepCAD、Text2CAD和Fusion360 Gallery[3, 13, 4, 23]）主要面向单零件建模或数据收集设计，因此无法评估装配层级推理、零件间关系或工程正确性。为填补这一空白，我们推出了assembench——一个基于Fusion360 Gallery装配语料库构建的基准测试。assembench并非通过收集新数据集来实现，而是通过精心筛选具有机械意义的装配体，并将其与通过半自动标注流程生成的统一自然语言装配规范进行配对，从而建立了标准化评估方案（附录F）。该基准测试根据以下三项原则构建：(1)**工程现实性**——选择具有明确工程语义的功能机械组件；(2)**结构复杂性**——要求存在多个相互作用的组件及非平凡的装配关系；(3)**参数化可构造性**——优先考虑可通过程序化CAD操作精确重建的装配结构。如表9所示，Assembench包含120个装配实例，涵盖多种机械结构及不同级别的装配复杂度。该基准测试专门用于评估本文提出的任务建模方法，要求模型能够协同合成组件、还原装配结构并满足工程约束条件。关于基准测试的具体构建过程、统计结果及代表性示例，详见附录F。

5.2 评估方案

我们在assembench提供的两种基准测试场景下对所有方法进行评估：**文本到CAD装配生成**与**图像-文本到CAD装配生成**。由于生产级CAD装配生成既需要可执行几何结构，又需正确的装配语义，我们采用三种互补的评估指标对各方法进行评估：**成功率（SR）**、**装配保真率（APR）**以及**基于VLM的工程评估**。

Chamfer距离。尽管Chamfer距离（CD）在以往的3D生成研究中被广泛采用，但我们特意将其排除在主要评估指标之外。如附录H所述，CD在装配评估中本质上是不可靠的：它既缺乏**装配结构可识别性**（命题3），也缺乏**装配排序一致性**（定理1）。具体而言，一个结构正确的装配体可能获得比缺失功能关键部件的装配体更差的CD评分（等式5）。因此，我们采用了能够直接评估结构正确性和工程质量的装配感知度量指标。

成功率（SR）。SR用于衡量基准测试实例中成功生成可执行CAD产物且无运行时错误的百分比。**汇编保留率（APR）**。APR用于衡量可执行CAD程序中保留预期汇编结构的百分比。汇编保留性通过基于LLM的代码评判器进行评估，其中得分至少为4分（满分5分）的预测结果被视为成功。**基于VLM的工程评估**。除可执行性和汇编保留性外，我们还采用“VLM作为评判器”的方法来评估工程质量。给定输入规范、参考渲染结果及生成的渲染结果，VLM会报告两项分数：**汇编**——用于衡量结构正确性；以及**综合**——用于衡量整体工程质量。

5.3 主要结果

表3总结了在仅文本和仅图像装配生成两种设置下，assembench的定量结果。我们对比了assembench与代表性CAD生成方法及前沿基础模型。总体而言，assembench始终实现了最高的成功率（SR）和装配保真率

models. Overall, ASSEMCAD consistently achieves the highest Success Rate (SR) and Assembly Preservation Rate (APR), demonstrating its ability to reliably generate production-ready CAD assemblies while maintaining competitive engineering quality under the proposed VLM-as-a-Judge.

Text-to-CAD Assembly Generation. Under the text-only setting, frontier foundation models achieve relatively high execution success but exhibit a substantial drop in APR, indicating that many executable CAD programs fail to preserve the intended assembly structure. This SR–APR gap reveals a fundamental limitation of code-centric approaches: without explicit assembly semantics, executable programs may produce valid geometry while violating structural constraints such as port compatibility, mate consistency, or engineering axioms.

In contrast, ASSEMCAD achieves a SR of 89.17% and an APR of 87.50%, showing that nearly every successfully generated example also preserves valid assembly semantics. This near-perfect alignment between SR and APR is directly attributable to two core components. First, the port- and mate-based assembly library enforces structural correctness through typed port compatibility checks and deterministic mate transforms, ensuring that each assembly relation is geometrically realizable rather than merely syntactically valid. Second, the axiom-grounded specification requires every mate to reference explicit engineering principles, preventing semantically inconsistent assembly relations (e.g., mating a fastener to a gear mesh port) from entering the realization pipeline. Together, these mechanisms guarantee that successfully constructed assemblies are not only executable but also structurally and semantically faithful to the input specification. Furthermore, ASSEMCAD obtains the highest Assembly score, demonstrating superior structural correctness compared with both specialized CAD generation methods and general-purpose foundation models.

Image-Text-to-CAD Assembly Generation. Providing rendered images tends to improve visual and overall engineering judgments for several models. However, it does not consistently improve SR or APR, indicating that visual conditioning alone is insufficient for preserving assembly semantics. Nevertheless, preserving assembly semantics remains challenging, as reflected by the persistent gap between SR and APR. This gap suggests that visual perception can improve component-level geometry but provides limited guidance for assembly-level reasoning, where functional interfaces, mate compatibility, and engineering constraints must be explicitly modeled.

By explicitly modeling structured assembly specifications grounded in engineering axioms, ASSEMCAD again achieves the highest SR, APR, and Assembly score. The assembly library ensures that port-level interfaces are validated against concrete B-Rep geometry (Definition 4), while axiom references attached to each mate provide engineering rationale that visual information alone cannot supply. This confirms that reliable assembly generation requires the combination of structured assembly representations and axiom-grounded reasoning, rather than relying solely on visual perception or code generation capabilities.

Although GPT-5.5 achieves the highest Overall score in the text-only setting and Gemini 3.5 Flash obtains the highest Overall score in the image-conditioned setting, these scores are computed only over successfully generated examples. Since ASSEMCAD successfully generates substantially more assemblies, it is evaluated on a considerably larger and more challenging set of instances, naturally making it more difficult to maintain the same average Overall score. Moreover, the assembly library’s multi-layer verification pipeline (port-geometry consistency, clash detection, and connectivity analysis) admits assemblies that other methods would fail to produce, including structurally complex instances with intricate mate configurations. Nevertheless, ASSEMCAD remains highly competitive while consistently producing production-ready assemblies across the full benchmark.

5.4 Framework Generalization

For Claude-4.8, AssemCAD dramatically improves both the APR and the Assembly Structure score under the text-only setting. Similarly, when combined with GPT-5.5, AssemCAD consistently improves assembly validity and structural quality under the image-conditioned setting.

This backbone-agnostic improvement can be attributed to the design of the assembly library and axiom system. The port- and mate-based library provides a deterministic realization layer that is independent of which foundation model generates the specification: typed port compatibility, deterministic mate transforms (Eq. 2), and port-geometry consistency checks (Definition 4) operate identically regardless of the upstream reasoning model. Similarly, the 62 engineering axioms serve as a shared semantic contract that constrains the specification space, reducing the burden on the foundation model from generating correct assembly code to selecting appropriate components, ports, and axiom-grounded mates. Consequently, the framework is largely orthogonal to the underlying foundation model and can be readily integrated with different reasoning backbones.

Table 4: Generalization across different foundation models.

Method	APR↑	Assembly↑	Overall↑
🌟 Claude-4.8	0.8	2.70	4.37
+ AssemCAD	82.50	3.38	4.53
🌀 GPT-5.5	64.2	3.17	5.20
+ AssemCAD	71.67	3.58	4.92

（APR），证明了其能够在所提出的 VLM -as-a-Judge框架下，可靠地生成可直接用于生产的CAD装配体，同时保持具有竞争力的工程质量。

文本到CAD装配生成。在纯文本输入条件下，前沿基础模型虽能实现相对较高的执行成功率，但其APR值会出现显著下降，表明许多可执行的CAD程序未能保留预期的装配结构。这一SR-APR差距揭示了代码中心化方法的根本局限性：在缺乏明确装配语义的情况下，可执行程序可能生成有效的几何模型，却会违反诸如端口兼容性、配对一致性或工程公理等结构约束。

相比之下，AssemCAD实现了89.17%的SR和87.50%的APR，表明几乎所有成功生成的示例均能保留有效的装配语义。SR与APR之间的这种近乎完美的对齐可直接归因于两个核心组件：首先，基于端口与配对的装配库通过类型化端口兼容性检查和确定性配对转换来强制执行结构正确性，确保每个装配关系在几何上是可实现的，而非仅在语法上有效；其次，基于公理的规范要求每个配对必须引用明确的工程原则，从而防止语义不一致的装配关系（例如将紧固件装配至齿轮啮合端口）进入实现流程。这些机制共同保证了成功构建的装配不仅可执行，而且在结构和语义上均忠实于输入规范。此外，AssemCAD获得了最高的Assembly评分，证明其在结构正确性方面优于专用CAD生成方法及通用基础模型。

图像-文本至CAD装配生成。提供渲染图像往往能提升多个模型的视觉及整体工程判断能力，但并未一致地改善SR或APR指标，这表明仅靠视觉条件化不足以保留装配语义。然而，保留装配语义仍面临挑战，这体现在SR与APR之间持续存在的差距上。该差距表明，视觉感知可改善部件级几何信息，但对于装配级推理却提供的指导有限——在装配级推理中，功能接口、配对兼容性及工程约束均需进行显式建模。

通过显式建模基于工程公理的结构化装配规范，AssemCAD再次实现了最高的SR、APR及装配评分。该装配库可确保端口级接口均经过具体B-Rep几何体的验证（定义4），而附加于每个配合关系的公理引用则提供了仅凭视觉信息无法提供的工程依据。这证实了可靠的装配生成需要结合结构化装配表示与基于公理的推理，而非单纯依赖视觉感知或代码生成能力。

尽管GPT-5.5在纯文本设置下取得了最高的综合得分，而Gemini 3.5 Flash在图像条件设置下也获得了最高的综合得分，但这些得分仅基于成功生成的示例计算得出。由于AssemCAD能够成功生成数量显著更多的装配体，其评估基于一个规模更大、更具挑战性的实例集，这自然使得维持相同的平均综合得分变得更加困难。此外，该装配库的多层验证流程（包括端口-几何一致性检查、冲突检测及连通性分析）能够生成其他方法无法生成的装配体，例如具有复杂匹配配置的结构化实例。尽管如此，AssemCAD仍保持了极强的竞争力，在整个基准测试中持续产出符合生产标准的装配体。

5.4 框架泛化

对于Claude-4.8，AssemCAD在纯文本设置下显著提升了APR和组装结构得分；同样地，当与GPT-5.5结合时，AssemCAD在图像条件设置下持续提升组装有效性与结构质量。这种不依赖于具体基础模型的改进可归因于装配库与公理系统的设计。基于端口和配对的库提供了一个确定性实现层，该层独立于何种基础模型生成规范：即类型化端口兼容性、确定性配对转换。

公式（等式2）与端口几何一致性检查（定义4）的运作方式完全相同，且不受上游推理模型的影响。类似地，62条工程公理充当了共享的语义契约，约束了规范空间，从而减轻了基础模型从生成正确汇编代码到选择合适组件、端口及基于公理的匹配项的负担。因此，该框架与底层基础模型基本互不干扰，可轻松集成于不同的推理骨干架构中。

表4：不同基础模型间的泛化能力。

方法	APR ↑、组装↑、总体↑		
🌟 Claude-4.8	0.8	2.70	4.37
+ AssemCAD	82.50	3.38	4.53
🌀 GPT-5.5	64.2	3.17	5.20
+ AssemCAD	71.67	3.58	4.92

References

- [1] Antoine Brière-Côté, Louis Rivest, and Roland Maranzana. Comparing 3d cad models: uses, methods, tools and perspectives. *Computer-Aided Design and Applications*, 9(6):771–794, 2012.
- [2] Joseph G Lambourne, Karl DD Willis, Pradeep Kumar Jayaraman, Aditya Sanghi, Peter Meltzer, and Hooman Shayani. Brepnet: A topological message passing system for solid models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12773–12782, 2021.
- [3] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6772–6782, 2021.
- [4] Gaurav Verma. *Autodesk fusion 360 black book*. BPB Publications, 2018.
- [5] Jeremy Wright, Marcus Boyd, Hasan Yavuz ÖZDERYA, Bruno Agostini, Michael Greminger, Seth Fischer, Justin Buchanan, Miguel Sánchez de León Peque, Martin Budden, Peter Boin, et al. Cadquery/cadquery: Cadquery 2.4. 0. *Zenodo*, 2024.
- [6] Felipe Machado, Norberto Malpica, and Susana Borromeo. Parametric cad modeling for open source scientific hardware: Comparing openscad and freecad python scripts. *Plos one*, 14(12):e0225795, 2019.
- [7] Haoyang Xie and Feng Ju. Text-to-cadquery: A new paradigm for cad generation with scalable large model capabilities. *arXiv preprint arXiv:2505.06507*, 2025.
- [8] Xueyang Li, Yu Song, Yunzhong Lou, and Xiangdong Zhou. Cad translator: An effective drive for text to 3d parametric computer-aided design generative modeling. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 8461–8470, 2024.
- [9] Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. Cadsmith: Multi-agent cad generation with programmatic geometric validation. *arXiv preprint arXiv:2603.26512*, 2026.
- [10] Ke Niu, Haiyang Yu, Zhuofan Chen, Zhengtao Yao, Weitao Jia, Xiaodong Ge, Jingqun Tang, Benlei Cui, Bin Li, and Xiangyang Xue. Cme-cad: Heterogeneous collaborative multi-expert reinforcement learning for cad code generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 39272–39281, 2026.
- [11] Tao Hu, Jiaxin Ai, Licheng Wen, Xueheng Li, Shu Zou, Siqi Li, Nianchen Deng, Xinyu Cai, Hongbin Zhou, Pinlong Cai, et al. Itercad: An iterative multimodal agent for visually-grounded cad generation and editing. *arXiv preprint arXiv:2606.13368*, 2026.
- [12] Jiaxin Ai, Tao Hu, Xuemeng Yang, Shu Zou, Hairong Zhang, Daocheng Fu, Yu Yang, Hongbin Zhou, Nianchen Deng, Pinlong Cai, et al. Comact: Reframing professional software manipulation via com-as-action paradigm. *arXiv preprint arXiv:2606.13239*, 2026.
- [13] Mohammad S Khan, Sankalp Sinha, Talha U Sheikh, Didier Stricker, Sk A Ali, and Muhammad Z Afzal. Text2cad: Generating sequential cad designs from beginner-to-expert level text prompts. *Advances in Neural Information Processing Systems*, 37:7552–7579, 2024.
- [14] Yujia Liu, Anton Obukhov, Jan Dirk Wegner, and Konrad Schindler. Point2cad: Reverse engineering cad models from 3d point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 3763–3772, 2024.
- [15] Maksim Kolodiazhnyi, Denis Tarasov, Dmitrii Zhemchuzhnikov, Alexander Nikulin, Ilya Zisman, Anna Vorontsova, Anton Konushin, Vladislav Kurenkov, and Danila Rukhovich. cadrille: Multi-modal cad reconstruction with online reinforcement learning. *arXiv preprint arXiv:2505.22914*, 2025.
- [16] Anthropic. Claude code. <https://docs.anthropic.com/en/docs/claude-code>, 2025. Accessed: 2026-06-30.
- [17] OpenAI. Introducing codex. <https://openai.com/index/introducing-codex/>, 2025. Accessed: 2026-06-30.
- [18] George Comanici et al. Gemini 2.5: Pushing the frontier with advanced reasoning and multimodality. *arXiv preprint arXiv:2507.06261*, 2025.

参考文献

- [1] Antoine Briere-Cote、Louis Rivest与Roland Maranzana。《三维CAD模型比较：应用、方法、工具及展望》。《计算机辅助设计与应用》，9(6): 771–794,2012年。
- [2] Joseph G Lambourne、Karl DD Willis、Pradeep Kumar Jayaraman、Aditya Sanghi、Peter Meltzer 和 Hooman Shayani。Brepnet: 一种用于实体模型的拓扑消息传递系统。载于*IEEE/ CVF 计算机视觉与模式识别会议论文集*，第12773–12782页，2021年。
- [3] Rundi Wu、Chang Xiao与Changxi Zheng。《Deepcad: 一种用于计算机辅助设计模型的深度生成网络》。载于*IEEE/ CVF 国际计算机视觉会议论文集*，第6772–6782页，2021年。
- [4] Gaurav Verma。《Autodesk Fusion 360 黑皮书》。BPB Publications，2018年。
- [5] Jeremy Wright、Marcus Boyd、Hasan Yavuz ozderya、Bruno Agostini、Michael Greminger、Seth Fischer、Justin Buchanan、Miguel Sanchez de Leon Peque、Martin Budden、Peter Boin 等。Cadquery/cadquery: Cadquery 2.4.0。Zenodo，2024年。
- [6] Felipe Machado、Norberto Malpica与Susana Borromeo。开源科学硬件的参数化CAD建模：对比OpenS-CAD与FreeCAD Python脚本。PLOS ONE，14(12): e0225795,2019年。
- [7] 谢浩阳、傅俊。《文本到CAD: 一种具备可扩展大模型能力的CAD生成新范式》arXiv预印本 arXiv: 2505.06507，2025年。
- [8] Li Xueyang、Song Yu、Lou Yunzhong与Zhou Xiangdong。《CAD Translator: 文本至三维参数化计算机辅助设计生成建模的有效驱动工具》。载于第32届ACM国际多媒体会议论文集，第8461–8470页，2024年。
- [9] Jesse Barkley、Rumi Loghmani与Amir Barati Farimani。Cadsmith: 支持程序化几何验证的多智能体CAD生成系统。arXiv预印本 arXiv:2603.26512，2026年。
- [10] Ke Niu、Haiyang Yu、Zhuofan Chen、Zhengtao Yao、Weitao Jia、Xiaodong Ge、Jingqun Tang、Benlei Cui、Bin Li 和 Xiangyang Xue。Cme-cad: 用于CAD代码生成的异构协作多专家强化学习。载于*IEEE/ CVF 计算机视觉与模式识别会议论文集*，第39272–39281页，2026年。
- [11] Tao Hu、Jiaxin Ai、Licheng Wen、Xueheng Li、Shu Zou、Siqu Li、Nianchen Deng、Xinyu Cai、Hongbin Zhou、Pinlong Cai, et al. Itercad: 一种用于视觉引导的CAD生成与编辑的迭代式多模态智能体。arXiv预印本 arXiv: 2606.13368,2026年。
- [12] Jiaxin Ai、Tao Hu、Xueming Yang、Shu Zou、Hairong Zhang、Daocheng Fu、Yu Yang、Hongbin Zhou、Nianchen Deng、Pinlong Cai, et al. Comact: 通过 “com-as-action” 范式重新构建专业软件操作。arXiv预印本 arXiv: 2606.13239,2026年。
- [13] Mohammad S Khan、Sankalp Sinha、Talha U Sheikh、Didier Stricker、Sk A Ali 和 Muhammad Z Afzal。Text2cad: 基于从初学者到专家级别的文本提示词生成序列性CAD设计。《神经信息处理系统进展》，第37卷: 7552–7579页，2024年。
- [14] 刘宇佳、Anton Obukhov、Jan Dirk Wegner与Konrad Schindler。Point2cad: 基于三维点云逆向工程CAD模型。载于*IEEE/ CVF 计算机视觉与模式识别会议论文集*，第3763–3772页，2024年。
- [15] Maksim Kolodiaznyi、Denis Tarasov、Dmitrii Zhemchuzhnikov、Alexander Nikulin、Ilya Zisman、Anna Vorontsova、Anton Konushin、Vladislav Kurenkov与Danila Rukhovich。cadrille: 基于在线强化学习的多模态CAD重建。arXiv预印本 arXiv:2505.22914，2025年。
- [16] Anthropic。Claude代码。<https://docs.anthropic.com/en/docs/claude-code>，2025年。访问日期: 2026-06-30。
- [17] OpenAI。介绍 Codex。<https://openai.com/index/introducing-codex/>，2025年。访问日期: 2026-06-30。
- [18] George Comanici 等，《Gemini 2.5: 借助先进推理与多模态技术拓展前沿》arXiv预印本 arXiv: 2507.06261,2025年。

- [19] GLM-4.5 Team. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.
- [20] Alibaba Qwen Team. Qwen3.7. <https://chat.qwen.ai/>, 2026.
- [21] Xiang Xu, Karl DD Willis, Joseph G Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman, and Yasutaka Furukawa. Skexgen: Autoregressive generation of cad construction sequences with disentangled codebooks. In *International Conference on Machine Learning*, pages 24698–24724. PMLR, 2022.
- [22] Xiang Xu, Pradeep Kumar Jayaraman, Joseph George Lambourne, Karl DD Willis, and Yasutaka Furukawa. Hierarchical neural coding for controllable cad model generation. In *International Conference on Machine Learning*, pages 38443–38461. PMLR, 2023.
- [23] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: a dataset and environment for programmatic cad construction from human design sequences. *ACM Trans. Graph.*, 40(4), July 2021.
- [24] Changjian Li, Hao Pan, Adrien Bousseau, and Niloy J. Mitra. Free2cad: parsing freehand drawings into cad commands. *ACM Trans. Graph.*, 41(4), July 2022.
- [25] Siyu Wang, Cailian Chen, Xinyi Le, Qimin Xu, Lei Xu, Yanzhou Zhang, and Jie Yang. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 7880–7888, 2025.
- [26] Jingwei Xu, Chenyu Wang, Zibo Zhao, Wen Liu, Yi Ma, and Shenghua Gao. Cad-mllm: Unifying multimodality-conditioned cad generation with mllm. *arXiv preprint arXiv:2411.04954*, 2024.
- [27] Yandong Guan, Xilin Wang, Ximing Xing, Jing Zhang, Dong Xu, and Qian Yu. Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward. *Advances in Neural Information Processing Systems*, 38:59765–59789, 2026.
- [28] Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-recode: Reverse engineering cad code from point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9801–9811, 2025.
- [29] Pradeep Kumar Jayaraman, Aditya Sanghi, Joseph G. Lambourne, Karl D. D. Willis, Thomas Davies, Hooman Shayani, and Nigel Morris. Uv-net: Learning from boundary representations. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11698–11707, 2021.
- [30] Haoxiang Guo, Shilin Liu, Hao Pan, Yang Liu, Xin Tong, and Baining Guo. Complexgen: Cad reconstruction by b-rep chain complex generation. *ACM Transactions on Graphics (TOG)*, 41(4):1–18, 2022.
- [31] Pradeep Kumar Jayaraman, Joseph G. Lambourne, Nishkrit Desai, Karl D. D. Willis, Aditya Sanghi, and Nigel J. W. Morris. Solidgen: An autoregressive model for direct b-rep synthesis, 2023.
- [32] Xiang Xu, Joseph Lambourne, Pradeep Jayaraman, Zhengqing Wang, Karl Willis, and Yasutaka Furukawa. Brepgen: A b-rep generative diffusion model with structured latent geometry. *ACM Trans. Graph.*, 43(4), July 2024.
- [33] Benjamin Jones, Dalton Hildreth, Duowen Chen, Ilya Baran, Vladimir G. Kim, and Adriana Schulz. Automate: a dataset and learning approach for automatic mating of cad assemblies. *ACM Trans. Graph.*, 40(6), December 2021.
- [34] Ram Anantha, Glenn A Kramer, and Richard H Crawford. Assembly modelling by geometric constraint satisfaction. *Computer-Aided Design*, 28(9):707–722, 1996.
- [35] Qiang Zou, Zhihong Tang, Hsi-Yung Feng, Shuming Gao, Chenchu Zhou, and Yusheng Liu. A review on geometric constraint solving. *arXiv preprint arXiv:2202.13795*, 2022.
- [36] Karl D.D. Willis, Pradeep Kumar Jayaraman, Hang Chu, Yunsheng Tian, Yifei Li, Daniele Grandi, Aditya Sanghi, Linh Tran, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Joinable: Learning bottom-up assembly of parametric cad joints. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15828–15839, 2022.
- [37] James Noeckel, Benjamin T. Jones, Karl Willis, Brian Curless, and Adriana Schulz. Mates2motion: Learning how mechanical cad assemblies work, 2023.

- [19] GLM-4.5 团队。Glm-4.5: 具备智能体、推理与编码 (ARC) 能力的基础模型。arXiv 预印本 arXiv:2508.06471, 2025年。
- [20] 阿里巴巴通义团队。Qwen3.7。https://chat.qwen.ai/, 2026年。
- [21] Xiang Xu, Karl DD Willis, Joseph G Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman 和 Yasutaka Furukawa。Skexgen: 基于解耦码本的CAD结构序列自回归生成方法。载于《国际机器学习会议》, 第24698–24724页。PMLR, 2022年。
- [22] Xiang Xu, Pradeep Kumar Jayaraman, Joseph George Lambourne, Karl DD Willis 和 Yasutaka Furukawa。《可控CAD模型生成的层次化神经编码》。载于《国际机器学习会议》, 第38443–38461页。PMLR, 2023年。
- [23] Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama 和 Wojciech Matusik。《Fusion 360 Gallery: 基于人类设计序列进行程序化CAD构建的数据集与环境》。ACM Trans. Graph., 第40卷第4期, 2021年7月。
- [24] 李长健、潘浩、Adrien Bousseau与Niloy J. Mitra。Free2cad: 将自由手绘图解析为CAD命令。ACM Trans. Graph., 41(4), 2022年7月。
- [25] Wang Siyu, Chen Cailian, Le Xinyi, Xu Qimin, Xu Lei, Zhang Yanzhou与Yang Jie。Cad-GPT: 利用空间推理增强的多模态大语言模型合成CAD构建序列。载于《AAAI人工智能大会论文集》, 第39卷, 第7880–7888页, 2025年。
- [26] Xu Jingwei, Wang Chenyu, Zhao Zibo, Liu Wen, Ma Yi 及 Gao Shenghua。Cad-MLLM: 将多模态条件下的CAD生成与MLLM统一。arXiv 预印本 arXiv:2411.04954, 2024年。
- [27] Guan Yandong, Wang Xilin, Xing Ximing, Zhang Jing, Xu Dong 及 Yu Qian。Cad-coder: 基于思维链与几何奖励的文本到CAD生成。《神经信息处理系统进展》, 第38卷: 59765–59789页, 2026年。
- [28] Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem与Djamila Aouada。《Cad-recode: 从点云逆向工程CAD代码》。载于IEEE/ CVF 国际计算机视觉会议论文集, 第9801–9811页, 2025年。
- [29] Pradeep Kumar Jayaraman, Aditya Sanghi, Joseph G. Lambourne, Karl D. D. Willis, Thomas Davies, Hooman Shayani, 和 Nigel Morris。Uv-net: 从边界表示中学习。载于2021年IEEE/ CVF Computer Vision 与 Pattern Recognition 会议 (CVPR), 第11698–11707页, 2021年。
- [30] 郭浩翔、刘世林、潘浩、刘阳、童新、郭贝宁。Complexgen: 通过B-Rep链复形生成实现CAD重建。《ACM Transactions on Graphics (TOG)》, 41(4): 1–18, 2022年。
- [31] Pradeep Kumar Jayaraman, Joseph G. Lambourne, Nishkrit Desai, Karl D. D. Willis, Aditya Sanghi与Nigel J. W. Morris。《Solidgen: 一种用于直接B-Rep合成的自回归模型》, 2023年。
- [32] Xiang Xu, Joseph Lambourne, Pradeep Jayaraman, Zhengqing Wang, Karl Willis 与 Yasutaka Furukawa。Brepgen: 一种具有结构化潜在几何特征的B-Rep生成扩散模型。ACM Trans. Graph., 43(4), 2024年7月。
- [33] Benjamin Jones, Dalton Hildreth, Duowen Chen, Ilya Baran, Vladimir G. Kim 和 Adriana Schulz。《Automate: 用于CAD装配件自动配对的数据集与学习方法》。ACM Trans. Graph., 第40卷第6期, 2021年12月。
- [34] Ram Anantha, Glenn A Kramer与Richard H Crawford。基于几何约束满足的装配建模。《计算机辅助设计》, 28(9): 707–722, 1996年。
- [35] Zou Qiang, Tang Zhihong, Feng Hsi-Yung, Gao Shuming, Zhou Chenchu与Liu Yusheng。《几何约束求解综述》。arXiv 预印本 arXiv:2202.13795, 2022年。
- [36] Karl D.D. Willis, Pradeep Kumar Jayaraman, Hang Chu, Yunsheng Tian, Yifei Li, Daniele Grandi, Aditya Sanghi, Linh Tran, Joseph G. Lambourne, Armando Solar-Lezama 和 Wojciech Matusik。《Joinable: 学习参数化CAD关节的自下而上组装》。载于2022年IEEE/ CVF 计算机视觉与模式识别会议 (CVPR), 第15828–15839页, 2022年。
- [37] James Noeckel, Benjamin T. Jones, Karl Willis, Brian Curless 和 Adriana Schulz。《Mates2motion: 学习机械卡盘组件的工作原理》, 2023年。

- [38] Yunsheng Tian, Jie Xu, Yichen Li, Jieliang Luo, Shinjiro Sueda, Hui Li, Karl D. Willis, and Wojciech Matusik. Assemble them all: Physics-based planning for generalizable assembly by disassembly. *ACM Transactions on Graphics*, 41(6).
- [39] Yunsheng Tian, Karl D. D. Willis, Bassel Al Omari, Jieliang Luo, Pingchuan Ma, Yichen Li, Farhad Javid, Edward Gu, Joshua Jacob, Shinjiro Sueda, Hui Li, Sachin Chitta, and Wojciech Matusik. Asap: Automated sequence planning for complex robotic assembly with physical feasibility. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4380–4386, 2024.
- [40] Dimitrios Mallis, Ahmet Serda Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-assistant: tool-augmented vllms as generic cad task solvers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7284–7294, 2025.
- [41] Benjamin T Jones, Zihan Zhang, Felix Hähnlein, Wojciech Matusik, Maaz Ahmad, Vladimir Kim, and Adriana Schulz. A solver-aided hierarchical language for llm-driven cad design. In *Computer Graphics Forum*, volume 44, page e70250. Wiley Online Library, 2025.

- [38] Yunsheng Tian、Jie Xu、Yichen Li、Jieliang Luo、Shinjiro Sueda、Hui Li、Karl D. Willis 及 Wojciech Matusik。《整合所有要素：基于物理的可泛化拆装装配规划》。《*ACM Transactions on Graphics*》，第41卷，第6期）。
- [39] 田云生、Karl D. D. Willis、Bassel Al Omari、罗继亮、马平川、李亦辰、Farhad Javid、Edward Gu、Joshua Jacob、Shinjiro Sueda、Li Hui、Sachin Chitta 及 Wojciech Matusik。《Asap：基于物理可行性实现复杂机器人装配的自动化序列规划》。载于2024年*IEEE国际机器人与自动化会议 (ICRA)*，第4380–4386页，2024年。
- [40] Dimitrios Mallis、Ahmet Serda Karadeniz、Sebastian Cavada、Danila Rukhovich、Niki Foteinopoulou、Kseniya Cherenkova、Anis Kacem 和 Djamila Aouada。Cad-Assistant：基于工具增强的VLLM作为通用CAD任务求解器。载于*IEEE/CVF 国际计算机视觉会议论文集*，第7284–7294页，2025年。
- [41] Benjamin T Jones、Zihan Zhang、Felix Hahnlein、Wojciech Matusik、Maaz Ahmad、Vladimir Kim 及 Adriana Schulz。《一种基于求解器辅助的LLM驱动CAD设计语言》。载于《*Computer Graphics Forum*》，第44卷，页码e70250。Wiley Online Library，2025年。

Appendix

Contents

A Engineering Axiom Set	16
A.1 Axiom Extraction Pipeline	17
A.2 Axiom Usage in the Pipeline	18
A.3 Example Axioms	18
B CAD Assembly Extension Library	18
B.1 Motivation	18
B.2 Architecture	18
B.3 Port Abstraction	19
B.4 Deterministic Mate Transform	19
B.5 Mate Types	19
B.6 Robust Boolean Operations	20
B.7 Port-Geometry Verification	20
B.8 Factory and Adapter System	20
B.9 Declarative Tree-Driven Assembly	21
C Built-in Parametric Factory Registry	21
D YAML Specification Format	21
E Failure Mode Catalog	22
F Benchmark Construction	22
F.1 Construction Pipeline	22
F.2 Prompt Generation	23
G Benchmark Visualization	24
H On the Unreliability of Chamfer Distance for Assembly Evaluation	24
H.1 Background and Definition	24
H.2 Information Loss under Point-Set Projection	24
H.3 CD Lacks Assembly Structure Identifiability	25
H.4 CD Lacks Assembly Ranking Consistency	25
H.5 Surface-Area Bias	26

A Engineering Axiom Set

Our axiom set $\mathcal{A}$ contains 62 axioms: 41 anchored to a raw pool of 139 textbook-extracted candidates, plus 21 synthesized to fill coverage gaps for component families present in the CAD libraries but absent from the source corpus. The axioms follow the MECE principle: mutually exclusive on scope

附录

目录

工程公理集	16
A.1 公理提取流水线	17
A.2 管道中的公理使用	18
A.3 示例公理	18
CAD装配扩展库	18
B.1 动机	18
B.2 架构	18
B.3 端口抽象	19
B.4 确定性棋子变换	19
B.5 茶类	19
B.6 鲁棒布尔运算	20
B.7 端口几何验证	20
B.8 工厂与适配器系统	20
B.9 声明式树驱动装配	21
C内置参数化工厂注册表	21
D YAML 规范格式	21
E失效模式目录	22
F基准测试构建	22
F.1 施工管线	22
F.2 提示词生成	23
G基准测试可视化	24
H：关于倒角距离在装配评估中的不可靠性	24
H.1 背景与定义	24
H.2 点集投影下的信息损失	24
H.3 CD缺乏装配结构可辨识度	25
H.4 CD的装配排名一致性不足	25
H.5 表面积偏差	26

A 工程公理集

我们的公理集A包含62条公理：其中41条基于139个从教科书提取的候选项构成的原始池，另有21条通过合成方式生成，以填补CAD库中存在但源语料库中缺失的组件族的覆盖空白。这些公理遵循MECE原则：在范围上互斥（即任意两条公理不涵盖相同的设计决策），并在CadQuery/cq_

(no two axioms cover the same design decision) and collectively exhaustive over the CadQuery/cq_warehouse/cq_gears expressible space. This section describes the three-phase extraction pipeline and lists representative axioms.

A.1 Axiom Extraction Pipeline

The axiom set is produced through a three-phase LLM-assisted knowledge distillation pipeline that transforms multilingual assembly textbooks into a versioned, auditable set of engineering axioms.

Phase 1: Two-Stage LLM Extraction. The source corpus consists of MinerU-digitized assembly textbooks (Chinese and English). The corpus is recursively segmented into section-scoped chunks by heading level. Two sequential LLM stages then process each chunk:

- **Stage A (Recall-focused Knowledge Unit Extraction).** For each chunk, an LLM extracts every atomic assembly-related claim as a *knowledge unit* (KU), classified by claim type (topology, sequence, geometry, process, material specification, safety). The prompt explicitly prioritizes recall over precision: claims are extracted verbatim without filtering or generalization. This stage produced 210 chunks yielding the initial KU pool.
- **Stage B (Precision-focused Axiom Canonicalization).** Each KU is individually assessed against strict axiom criteria: it must be (a) topological or sequential in nature, (b) component-class-scoped rather than product-specific, (c) free of numerical specifications, and (d) universally true rather than a recommendation. Accepted KUs are canonicalized into present-tense active voice, assigned to one of six categories (fastener, gear, bearing, base_frame, general_sequence, general_topology), tagged with controlled-vocabulary scope labels, and given a confidence score. Rejected KUs are logged with structured rejection reasons (not_universal, numerical_spec, not_assembly, too_vague, product_specific, process_only).

The two-stage split enables localized failure analysis: a missing axiom is traceable to either a Stage-A recall miss or a Stage-B precision reject. The pipeline uses deterministic settings (seed=42, temperature=0.0) for reproducibility, and a schema validator enforces a controlled vocabulary of part types, feature types, relation types, and sequence constraints. This phase produced 139 raw candidate axioms.

Phase 2: LLM Review and Filtering. A second LLM pass reduces the raw pool through: (1) clustering by deduplicated content key, (2) per-cluster keep/merge/drop decisions, and (3) cross-cluster semantic deduplication using embeddings. Six explicit exclusion rules are applied: axioms that merely restate geometric definitions, contain numerical specifications, are product-catalog-specific, are historical in nature, duplicate a sibling, or contradict a stronger sibling are dropped. Surviving axioms are reworded into canonical MUST/MUST NOT/SHOULD phrasing.

Phase 3: MECE Finalization. The final pass is an interactive audit enforcing four properties:

1. **Mutual exclusivity:** every pair of axioms within each scope is checked for overlap.
2. **Universality:** each axiom is validated against demonstrator assembly examples.
3. **Coverage:** every port type and component class in the CAD extension library (Appendix B) must be covered by at least one axiom.
4. **DAG consistency:** precedes/follows relationships among axioms must form a directed acyclic graph.

Where the textbook corpus lacked coverage for component families present in the CAD libraries (e.g., certain bearing configurations), *synthesized axioms* are created with explicit provenance markers and engineering justification.

The disposition of the 139 raw axioms was: 37 kept directly, 16 merged (subsumed by existing axioms), 60 dropped as manufacturing-specific (out of scope), 19 dropped as overly narrow, and 7 dropped as trivial or tautological. An additional 4 axioms were broadened beyond their raw sources (counted as anchored but with synthesized generalizations), yielding 41 raw-anchored axioms. Finally, 21 axioms were synthesized to cover component families present in the CAD libraries but absent from the textbook corpus (e.g., washers, setscrews, keyways, helical/bevel/worm gears, sprockets, pulleys, couplings, plain bearings), each marked with explicit provenance and engineering justification. The final set contains $41 + 21 = 62$ axioms.

warehouse/cq_gears可表达空间内具有整体完备性。本节将介绍三阶段提取流程，并列出发性公理。

A.1 公理提取流水线

公理集是通过一个三阶段LLM辅助知识蒸馏流程生成的，该流程可将多语言汇编教材转化为版本化、可审计的工程公理集。

第一阶段：两阶段大语言模型提取。源语料库由MinerU数字化的装配教材（中文与英文）组成。该语料库根据标题层级递归分割为章节范围内的片段；随后，两个连续的大语言模型处理阶段会对每个片段进行处理：

- **阶段A（召回导向的知识单元提取）。**对于每个文本块，大语言模型会提取所有与原子组装相关的声明作为*知识单元*（KU），并按声明类型（拓扑结构、序列、几何结构、工艺流程、材料规格、安全要求）进行分类。提示词明确优先考虑召回而非精确度：声明将逐字提取，未经任何过滤或泛化处理。该阶段共生成210个文本块，形成初始的 KU 池。
- **阶段B（精度导向的公理规范归一化）。**每个 KU 均需根据严格的公理标准进行单独评估：其必须满足以下条件：(a)本质为拓扑或序列型；(b)属于组件类别范围而非产品特定；(c)不含数值规格；(d)具有普遍适用性而非仅为建议。通过审核的KU将被归一化为现在时主动语态，分配至六类之一（紧固件、齿轮、轴承、基座框架、通用序列、通用拓扑），标注受控词汇范围标签，并赋予置信度评分；而未通过审核的KU则会记录其结构化驳回原因（非普遍适用、含数值规格、非装配部件、表述过于模糊、产品特定、仅适用于流程）。

两阶段拆分机制可实现局部故障分析：缺失公理可追溯至Stage-A召回失败或Stage-B精确拒绝。该流程采用确定性设置（seed=42，temperature=0.0）以确保可复现性，并通过模式验证器强制执行受控的部件类型、特征类型、关系类型及序列约束词汇表。此阶段共生成了139条原始候选公理。

第二阶段：LLM复核与过滤。第二次LLM处理会通过以下步骤减少原始候选池：(1) 根据去重内容键进行聚类；(2) 对每个聚类做出保留、合并或删除的决策；(3) 利用嵌入向量进行跨聚类语义去重。本阶段应用了六条明确的排除规则：若公理仅重复陈述几何定义、包含数值规格、特定于产品目录、具有历史性质、与兄弟公理重复，或与更强的兄弟公理相矛盾，则予以剔除。保留的公理将被重新表述为标准的“必须”、“禁止”或“应”表述形式。

第三阶段：MECE 最终确认。最后一轮为交互式审计，旨在强制执行四项属性：

1. **互斥性：**在每个作用域内，对每一对公理进行重叠检查。
2. **普适性：**每条公理均通过示例组件实例进行验证。
3. **覆盖范围：**CAD扩展库（附录B）中的所有端口类型和组件类均须至少被一个公理所覆盖。
4. **DAG一致性：**公理间的前驱/后继关系必须构成一个有向无环图。

当教科书语料库未能涵盖CAD库中出现的组件族（例如某些轴承配置）时，会合成公理并为其添加明确的来源标识及工程依据。

139条原始公理的处理结果如下：37条直接保留；16条合并（被现有公理所涵盖）；60条因属于制造特定内容而剔除（超出范围）；19条因过于狭隘而剔除；7条因属平凡或同义关系而剔除。另有4条公理从其原始来源出发进行了拓展（视为锚定但包含综合的广义表述），由此生成了41条原始锚定公理。最后，通过综合21条公理，以覆盖CAD库中存在但教科书语料库中缺失的组件类别（例如：垫圈、紧定螺钉、键槽、螺旋/锥形/蜗轮齿轮、链轮、滑轮、联轴器、滑动轴承），每条公理均标注了明确的来源及工程依据。最终集合包含 $41 + 21 = 62$ 条公理。

A.2 Axiom Usage in the Pipeline

Each axiom serves as a formal justification throughout the pipeline. In the Assembly Specification (Definition 1), every mate $m_j \in \mathcal{M}$ references one or more axioms via its $\mathcal{A}$ field. These references are: (1) attached during semantic decomposition (Algorithm 1), (2) used to guide component synthesis through provenance-aware prompts (Algorithm 2), and (3) propagated to the final verification report, creating an auditable chain from the output STEP file back through the assembly tree to the engineering principle and ultimately to the textbook source.

A.3 Example Axioms

Table 5 lists representative axioms from the final set, illustrating the range of engineering knowledge encoded across different categories. Each axiom is stated in canonical MUST/MUST NOT phrasing and is scoped to a specific component class or assembly relation.

Table 5: Representative engineering axioms from the final axiom set.

ID	Axiom Statement
F-01	An assembly is a set of parts related by pairwise connections; every connection between two parts removes one or more degrees of freedom from their relative motion.
F-03	Every pairwise connection is expressed as one or more geometric constraints between named features (faces, edges, axes, or points) of the two parts.
C-02	A coaxial constraint between two cylindrical axes makes them collinear, removing two translational and two rotational DOF and leaving one rotational and one translational DOF along the shared axis.
K-01	A revolute joint constrains two parts to a single rotational DOF about a fixed axis.
G-02	A spur-gear pair has parallel axes and cylindrical pitch surfaces; its teeth are straight and parallel to the gear axis.
T-01	A threaded fastener creates a tensile preload along its shank that clamps the joined parts together in compression between the fastener’s bearing surfaces.

B CAD Assembly Extension Library

The assembly realization pipeline described in Section 4 relies on a custom CadQuery extension library, `cad-assembly-ext`, that introduces a *port-based mating system* for deterministic mechanical assembly. This appendix details its architecture, core abstractions, and the design decisions motivated by the requirements of LLM-driven CAD generation.

B.1 Motivation

Vanilla CadQuery provides only raw `cq.Location`-based placement for multi-part assemblies, offering no semantic mating, no type safety, and no validation layer. This is insufficient for an LLM-driven pipeline for three reasons: (1) LLMs frequently produce structurally plausible but geometrically incorrect assemblies (e.g., bolts mated to gear meshes), requiring compile-time type checking on assembly operations; (2) boolean operations in the OpenCascade kernel silently produce disconnected compounds when solids share only boundary contact, a failure mode that LLM-generated code encounters systematically; and (3) LLM-synthesized components may contain *phantom features*—ports declared at locations where the corresponding geometric operation (e.g., a bore cut) silently failed—requiring geometric evidence verification beyond symbolic annotations.

B.2 Architecture

The library follows a three-phase data flow that mirrors the assembly realization pipeline:

A.2 管道中的公理使用

每条公理在整个流程中均发挥着形式化论证的作用。在“装配规范”（定义1）中，每个配对项 m_j M 通过其 A 字段引用一条或多条公理。这些引用关系包括：(1) 在语义分解阶段进行关联（算法1）；(2) 通过感知来源的提示词指导组件合成（算法2）；(3) 将引用信息传递至最终验证报告，从而构建一条可审计的链路——从输出的STEP文件出发，经由装配树，最终追溯至工程原理乃至教科书源文件。

A.3 示例公理

表5列出了最终集中的代表性公理，展示了跨不同类别编码的工程知识范围。每条公理均采用标准的“必须”/“不得”表述方式，并限定于特定组件类或装配关系。

表5：最终公理集中的代表性工程公理。

身份证	公理陈述
F-01	装配体是指由若干部件通过成对连接而构成的集合；部件之间每一对连接都能消除其相对运动中的一个或多个自由度。
F-03	每对配对连接均可表示为两个对象之间一个或多个几何约束，这些约束作用于两者的命名特征（面、边、轴或点）之上。
C-02	两个圆柱轴之间的同轴约束使它们共线，从而消除了两个平移 DOF 和两个旋转 DOF，仅保留沿共享轴上的一个旋转 DOF 和一个平移 DOF。
K-01	转动关节可将两个部件约束为绕固定轴的单一旋转 DOF。
G-02	直齿圆柱齿轮副的轴线相互平行，其齿面为圆柱形节线；其齿形为直线形，且与齿轮轴线平行。
T-01	螺纹紧固件在其杆部上可产生拉伸预载荷，该预载荷能在紧固件的承载面上使被连接部件相互压紧。

B CAD装配扩展库

第4节所述的装配实现流水线依赖于一个自定义的CadQuery扩展库——`cad-assembly-ext`，该库引入了基于端口的配合系统，用于实现确定性的机械装配。本附录详细阐述了其架构、核心抽象概念以及基于LLM驱动CAD生成需求所做出的设计决策。

B.1 动机

Vanilla CadQuery 仅提供原始的 `cq` 功能：对于多部件装配体，其支持基于位置的布局，但并未提供语义匹配、类型安全或验证层。这在构建基于大语言模型（LLM）的流程时是不够的，原因有三：(1) 大语言模型生成的装配体常在结构上看似合理，但在几何上存在错误（例如，螺栓与齿轮齿合面相配），因此需要对装配操作进行编译时的类型检查；(2) OpenCascade 内核中的布尔运算在实体仅存在边界接触的情况下，会静默地生成不连通的复合体——这种故障模式在由 LLM 生成的代码中具有系统性发生倾向；(3) 由 LLM 合成的组件可能包含“幻影特征”——即在相应几何操作（如钻孔）静默失败的位置声明了端口——这就要求必须进行超越符号化注释的几何证据验证。

B.2 架构

该库遵循三阶段数据流，其结构与汇编实现流水线相呼应：

Modeling Phase. Parts are constructed using standard CadQuery operations augmented with nine extension methods (e.g., `c bore_hole_with_port`, `shaft_port`, `bore_port`) that atomically create geometry *and* register a typed port annotation in a single call. This co-location prevents geometry–metadata drift.

Assembly Phase. `AssemblyExt.add()` extracts ports from each part’s context. The `.mate()` method resolves port references (string format “part_name:port_name”), checks type compatibility via a predefined compatibility matrix, and dispatches to one of seven mate-type functions that each compute a closed-form rigid-body transform.

Export Phase. `.build()` produces a standard `cq.Assembly` object; `.export_step()` writes STEP files with preserved assembly hierarchy.

B.3 Port Abstraction

The port abstraction is the central data structure that enables typed, semantically meaningful assembly interfaces. Each port encodes not only a spatial coordinate frame but also a type label and type-specific parameters, allowing the assembly engine to enforce compatibility constraints before geometric computation.

A *port* is a typed geometric attachment point:

$$\pi = (\text{name}, t \in \mathcal{T}_{\text{port}}, L \in SE(3), \phi),$$

where t is one of 12 port types (BOLT_SEAT, SHAFT, BORE, GEAR_MESH, THREAD_INTERNAL, THREAD_EXTERNAL, PRESS_FIT, FLAT_FACE, CLEARANCE_HOLE, CBORE_SEAT, HOLE, SNAP_FIT), L is the port’s coordinate frame, and ϕ contains type-specific parameters (e.g., diameter, depth, thread specification).

A compatibility matrix $\mathcal{C} \subseteq \mathcal{T}_{\text{port}} \times \mathcal{T}_{\text{port}}$ defines which port-type pairs may be mated, providing semantic guard rails that catch category errors (e.g., attempting to mate a fastener with a gear mesh) before geometric computation.

B.4 Deterministic Mate Transform

The central mathematical operation is a closed-form rigid-body transform, as given in Eq. (2) of the main text. Given a base port π_b (already placed in world coordinates) and an incoming port π_c (in the incoming part’s local frame), the transform that places the incoming part is $T = L_b \cdot R_{\text{flip}} \cdot R_\alpha \cdot L_c^{-1}$, where R_{flip} is a 180° rotation about the local x -axis (making z -axes anti-parallel for face-to-face contact), and R_α is an optional user-specified rotation about the shared z -axis. This single matrix multiplication—with no iterative constraint solving—produces bit-identical results across runs, satisfying Proposition 2.

B.5 Mate Types

The library implements seven mate types, each extending the base transform (Eq. 2) with type-specific geometric semantics and parameter validation. Table 6 summarizes their implementations.

建模阶段。零件通过标准的CadQuery操作构建而成，这些操作还增强了九种扩展方法（例如：`cbore_hole_with_port`、`shaft_port`、`bore_port`），这些方法可在单次调用中原子性地创建几何体并在其中注册带类型标注的端口注释。这种共定位机制可防止几何体与元数据之间的漂移。

装配阶段。`AssemblyExt.add()`方法从每个部件的上下文中提取端口；`.mate()`方法会解析端口引用（字符串格式“`part_name: port_name`”），通过预定义的兼容性矩阵检查类型兼容性，并将调用分发至七种匹配类型函数之一，每种函数均可计算出闭式刚体变换。

导出阶段。`.build()`方法会生成标准的cq装配对象；`.export_step()`方法则会写入保留装配层级结构的STEP文件。

B.3 端口抽象

端口抽象是实现带类型标注且具有语义意义的装配接口的核心数据结构。每个端口不仅编码了空间坐标系，还编码了类型标签及类型特定参数，从而使装配引擎能够在几何计算之前强制执行兼容性约束。

`Aport` 是一种带类型标注的几何附着点：

$$\acute{e} = (name, t, \mathcal{T}_{port}, L, SE(3), \acute{H})$$

其中 t 为12种端口类型之一（`BOLT_SEAT`、`shaft`、`BORE`、`GEAR_MESH`、`THREAD_INTERNAL`、`THREAD_EXTERNAL`、`PRESS_FIT`、`FLAT_FACE`、`CLEARANCE_HOLE`、`CBORE_SEAT`、`HOLE`、`SNAP_FIT`）， L 为端口的坐标系，而 $\acute{H}$ 包含类型特定参数（例如：直径、深度、螺纹规格）。

兼容性矩阵 $C: \mathcal{T}_{port} \times \mathcal{T}_{port}$ 定义了哪些端口类型对可进行匹配，从而提供语义约束机制，在几何计算前捕获类别错误（例如，尝试将紧固件与齿轮啮合进行匹配）。

B.4 确定性配对变换

核心数学运算为闭式刚体变换，如正文第2式所示。给定基座端口 π_b （已置于世界坐标系中）与输入端口 π_c （位于输入部件的局部坐标系内），用于定位输入部件的变换矩阵为 $T = L_b \cdot R_{flip} \cdot R_\alpha \cdot L_c^{-1}$ ，其中 R_{flip} 表示绕局部 x 轴进行 180° 旋转（使 z 轴在面对面接触时呈反平行关系），而 R_α 则是可选的用户指定绕共享 z 轴的旋转。这一单次矩阵乘法——无需迭代约束求解——可在不同运行中产生完全一致的结果，满足命题2。

B.5 茶类

该库实现了七种配对类型，每种类型均扩展了基础变换（等式2），并附加了特定类型的几何语义和参数验证功能。表6总结了它们的实现方式。

Table 6: Mate type implementations with their geometric semantics.

Mate Type	Semantics
face_to_face	Coplanar normals with optional in-plane offsets (u, v) and normal displacement d_n .
coaxial	Anti-parallel z -axes with optional depth offset along the shared axis.
coaxial_face	Axis alignment with contact; used for bolt-in-counterbore placement.
gear_mesh	Validates matching module m and pressure angle; computes center distance $(z_1 + z_2) \cdot m/2$ and applies phase correction when $(z_1 + z_2)$ is odd.
press_fit	Validates shaft diameter $\geq$ hole diameter (interference condition), then delegates to coaxial.
thread_engage	Validates thread specification compatibility, then delegates to coaxial with depth.
snap_to_face	Parametric (u, v) positioning on a planar face.

B.6 Robust Boolean Operations

To address the systematic failure mode where CadQuery’s `union()` silently produces disconnected compounds under face-only contact, the library provides `SAFEUNION` (Proposition 1), which employs a three-strategy cascade: (1) standard union when volumetric overlap exists; (2) glue-based fusion when bounding boxes touch; and (3) automatic micro-extension along the shortest gap direction when neither applies. A post-condition check asserts exactly one output solid, raising an error with diagnostic bounding-box information rather than propagating a disconnected compound.

B.7 Port-Geometry Verification

The library verifies that declared ports correspond to actual geometric features using OpenCascade’s `BRepClass3d_SolidClassifier`, implementing the port-geometry consistency checks defined in Definition 4:

- **Bore check:** samples axial points along the declared bore direction (must classify as OUT indicating cavity) and radial ring points (at least one must classify as IN indicating surrounding material).
- **Shaft check:** samples axial points at quarter and three-quarter shaft length (must classify as IN) and a radial point outside the shaft diameter (must classify as OUT).
- **Flat face check:** iterates all B-Rep faces on the solid to find one with collinear normal (within 10^{-2} rad) passing within 1 mm of the port origin.

Verification failures produce structured error messages with natural-language hints designed for LLM-consumable repair feedback, directly supporting the iterative generation-repair loop in Algorithm 2.

B.8 Factory and Adapter System

The library includes 8 parametric base-frame factories (Table 7) and adapter modules for two third-party libraries, covering fasteners, bearings, and gears:

- **Fastener adapters:** wrap `cq_warehouse` screws, nuts, and washers with 5, 3, and 3 ports respectively, including thread ports, head seats, and bearing faces.
- **Gear adapters:** wrap `cq_gears` spur gears with bore, mesh point (at pitch radius with transmission parameters in ϕ), and face ports.
- **Bearing adapters:** wrap `cq_warehouse` bearings with bore, outer race (`PRESS_FIT` type), and face ports.

A user `.*` factory namespace enables LLM-synthesized custom factories via dynamic import, supporting the open-world component synthesis described in Section 4.

表6: Mate类型实现及其几何语义。

类别	语义学
面对面	共面法线，可选择具有面内偏移量 (u, v) 和法向位移 d_n 。
同轴的	反平行 z 轴，可沿共享轴选择性地设置深度偏移。
同轴面	带接触的轴对齐；用于螺栓安装用沉头孔定位。
齿轮啮合	验证匹配模块 m 与压力角；计算中心距 $(z_1+z_2) \cdot m/2$ ，并在 (z_1+z_2) 为奇数时进行相位校正。
压配	若验证轴径 $\geq$ 孔径（过盈状态），则转为同轴判定。
线程参与	验证线程规格兼容性，随后委托给“同轴且有深度”处理。
snap_to_face	平面面上的参数化 (u, v) 定位。

B.6 鲁棒布尔运算

为解决CadQuery的`union()`函数在仅接触面条件下会静默生成不连通复合体的系统性故障模式，该库提供了`safeunion`函数（命题1），该函数采用三策略级联机制：(1) 当存在体积重叠时，执行标准合并；(2) 当包围盒相交时，执行基于粘合的融合；(3) 当上述两种情况均不适用时，沿最短间隙方向自动进行微扩展。后置条件检查会确保输出恰好为一个实体，并通过提供诊断用包围盒信息来抛出错误，而非传播不连通的复合体。

B.7 端口几何验证

该库利用OpenCascade的BRepClass3d_SolidClassifier对声明的端口与实际几何特征进行验证，实现了定义4中所规定的端口-几何一致性检查：

- **孔径检查**：沿声明的孔径方向采集轴向点样本（必须分类为“OUT”，表示存在空腔），并采集径向环形点样本（至少有一个点需分类为“IN”，表示周围材料）。
- **轴心检查**：在轴长度的四分之一和四分之四处采集轴向点样本（必须分类为“IN”），并在轴直径外侧采集径向点样本（必须分类为“OUT”）。
- **平面面检查**：遍历实体上的所有B-Rep面，以寻找一个法线共线（误差在 10^{-2} 弧度以内）且法线与端口原点的距离不超过1毫米的面。

验证失败会生成结构化的错误消息，其中包含针对LLM可消耗修复反馈设计的自然语言提示，直接支持算法2中的迭代生成-修复循环。

B.8 工厂与适配器系统

该库包含8个参数化基框架工厂（表7）以及针对两个第三方库的适配器模块，涵盖紧固件、轴承和齿轮：

- **紧固件适配器**：用于包裹cq_warehouse螺钉、螺母及垫圈，分别配备5个、3个和3个端口，包括螺纹端口、头部座及轴承面。
- **齿轮适配器**：用于缠绕带孔的直齿圆柱齿轮，其啮合点位于节圆半径处（传动参数为 $\check{p}$ ），并设有端面端口。
- **轴承适配器**：用于包裹具有孔径、外圈（压配型）及端面端口的cq_warehouse轴承。

用户*.factory命名空间通过动态导入功能支持LLM合成的自定义工厂，从而实现第4节所述的开放世界组件合成。

B.9 Declarative Tree-Driven Assembly

The function `assemble_from_tree()` accepts a JSON specification containing `parts_index` (ordered dictionary of part nodes) and `mates` (ordered list of mate declarations). It materializes all parts via factory dispatch, adds the first part as the base, then iterates mates in declaration order—each calling `AssemblyExt.mate()`. This declarative interface maps naturally to the structured Assembly Specification (Definition 1), enabling the LLM to output structured data rather than imperative CadQuery code.

C Built-in Parametric Factory Registry

The assembly library ships with 13 built-in parametric factories that cover the most commonly occurring mechanical components in the benchmark assemblies. Table 7 summarizes each factory together with its key parameters and the typed ports it declares. These factories serve as the default component source during assembly realization; components not covered by the registry are synthesized on demand via the LLM (Algorithm 2).

Table 7: The 13 built-in parametric factories with their key parameters and declared ports.

Factory	Key Parameters	Ports
<code>rect_plate</code>	L, W, T	<code>top_face</code> , <code>bottom_face</code> , $4 \times$ <code>side_face</code>
<code>l_bracket</code>	L_1, L_2, W, T	<code>face_a</code> , <code>face_b</code> , <code>web_face</code>
<code>u_channel</code>	L, W, H, T	<code>floor_face</code> , $2 \times$ <code>wall_face</code>
<code>boss</code>	D, H , bore	<code>top_face</code> , <code>bore_port</code>
<code>bolt_circle_flange</code>	D, H , <code>bolt_d</code> , n	<code>top_face</code> , <code>bottom_face</code> , <code>bolt_circle</code>
<code>housing_block</code>	L, W, H , bore	6 faces, <code>bore_port</code>
<code>stepped_shaft</code>	segments	per-segment <code>shaft_seats</code>
<code>coupling_hub</code>	D, L , bore	<code>shaft_seat</code> , face
<code>fastener.bolt</code>	d, L , head	thread, <code>head_face</code>
<code>fastener.nut</code>	d , width	thread_internal, face
<code>fastener.washer</code>	d_i, d_o, T	<code>top_face</code> , <code>bottom_face</code>
<code>gear.spur</code>	m, z , width, bore	<code>mesh_point</code> , bore
<code>bearing.ball</code>	d_i, d_o, W	<code>inner_ring</code> , <code>outer_ring</code>

D YAML Specification Format

The Assembly Specification (Definition 1) is serialized as a YAML document that the LLM produces during semantic decomposition. The format declares parts with their factory references and parameters, followed by mates that specify port endpoints, mate types, and axiom references. The following listing shows a representative specification for a single-stage spur gear reducer.

```
assembly:
  id: gear_reducer_v1
  description: "Single-stage spur gear reducer"
  parts:
    - id: housing
      factory: base_frame.housing_block
      params: {length: 120, width: 80, height: 60, bore_d: 25}
    - id: input_gear
      factory: gear.spur
      params: {module: 2, teeth: 20, width: 15, bore_d: 12}
    - id: output_gear
      factory: gear.spur
      params: {module: 2, teeth: 40, width: 15, bore_d: 16}
  mates:
    - id: m_input_shaft
      mate_type: coaxial
      base: {ref: housing, port: bore_port}
```


B.9 声明式树驱动装配

函数 `assemble_from_tree()` 接受一个包含 `'parts_index'`（部件节点的有序字典）和 `'mates'`（配对声明的有序列表）的 JSON 规范。该函数通过工厂分发机制实现所有部件的构建，首先将第一个部件作为基底，然后按声明顺序迭代遍历 `'mates'`——每次调用 `'AssemblyExt.mate()'` 方法。这种声明式接口自然映射到结构化的 Assembly Specification（定义1），使得大语言模型能够输出结构化数据而非命令式 CadQuery 代码。

C 内置参数化工厂注册表

该装配库随附13个内置参数化工厂，这些工厂涵盖了基准测试装配中最常见的机械部件。表7汇总了各工厂及其关键参数以及其声明的类型化端口。这些工厂在装配实现过程中充当默认组件源；未被注册表覆盖的组件则通过LLM（算法2）按需合成。

表7：13个内置参数化工厂及其关键参数与声明端口。

工厂	关键参数	端口
矩形板、L型	L, W, T L_1, L_2, W, T	顶面、底面、4 × 侧面 <code>face_a</code> 、 <code>face_b</code> 、 <code>web_face</code>
支架、U型槽	L, W, H, T	地板面，2 × 墙面 <code>top_face</code> 、 <code>bore_port</code>
螺栓、圆环法兰、机座块、阶梯轴、联轴器、毂、紧固件、螺栓	D, H ，孔径 D, H ，螺栓 d 、 n L, W, H ，孔径 部分 D, L ，孔径 d, L ，头部	顶面、底面、螺栓圆环 6个面， <code>bore_port</code> 按段划分的轴座 轴座，面 线程、头部面部
紧固件：螺母	d ，宽度	线程内部，面
紧固件、垫圈、直齿轴承、滚珠	d_i, d_o, T m, z ，宽度、孔径 d_i, d_o, W	顶面、底面 网格点、孔径 内环、外环

D YAML 规范格式

装配规范（定义1）被序列化为 YAML 文档，由大语言模型在语义分解过程中生成。该格式声明了各部件及其工厂引用和参数，随后是用于指定端口端点、配对类型及公理引用的配对项。下文列表展示了单级直齿减速器的典型规范示例。

议会:

ID: `gear_reducer_v1`

描述: “单级直齿圆柱齿轮减速器”

部:

- ID: 住房
工厂: `base_frame.housing_block`
参数: {长度: 120, 宽度: 80, 高度: 60, 孔径_d: 25}
- ID: `input_gear`
工厂: 齿轮——直齿
参数: {模块: 2, 齿数: 20, 宽度: 15, 孔径: 12}
- ID: `output_gear`
工厂: 齿轮——直齿
参数: {模块: 2, 齿数: 40, 宽度: 15, 孔径: 16}

伙伴:

- ID: `m_input_shaft`
mate_type: 同轴
基础: {ref: housing, port: `bore_port`}

```

    incoming: {ref: input_gear, port: bore}
    axioms: [C-02, P-01]
- id: m_gear_mesh
  mate_type: gear_mesh
  base: {ref: input_gear, port: mesh_point}
  incoming: {ref: output_gear, port: mesh_point}
  axioms: [G-02]

```

E Failure Mode Catalog

During development and evaluation, we systematically cataloged recurring failure modes encountered in LLM-driven CAD assembly generation. Table 8 summarizes the five most representative failure modes, their root causes, observable symptoms, and the specific validation gate in ASSEMBENCH that detects and prevents each failure from propagating to the final assembly.

Table 8: Systematic catalog of identified failure modes, their root causes, and the system component that addresses each.

Failure Mode	Root Cause	Symptom	Gate
Phantom bore	Workplane("XZ") sign trap causes .cut() to miss solid	Axle transfixes un-drilled arm ($\pi r^2 L$ clash)	Port-geom.
Disjoint compound	union(glue=False) on face-contact	Part renders as floating sub-pieces	safe_union
All-coincident parts	Zero-offset face_to_face to same port	Multiple parts at same location	Overlap validator
Port-frame surprise	Auto-derived <i>u/v</i> axes on oblique normal	Offset in unexpected world direction	Strict <i>x_dir</i>
Semantic mis-selection	LLM picks wrong factory for role	Geometry correct but functionally wrong	Visual reflection

F Benchmark Construction

F.1 Construction Pipeline

ASSEMBENCH is constructed upon the assembly subset of the Fusion360 Gallery dataset [23]. Rather than manually annotating CAD assemblies, we develop a semi-automatic annotation pipeline that combines rule-based parsing with vision-language understanding to produce standardized natural-language assembly specifications.

Given an assembly instance, we first parse its native Fusion360 assembly JSON file using a rule-based parser. The parser extracts structured assembly information, including component hierarchy, assembly occurrences, joints, contacts, and other assembly metadata. This structured representation provides an explicit description of the assembly topology and engineering relationships.

To complement the structural information, each assembly is rendered from two canonical viewpoints using the original CAD geometry. The rendered images preserve the global appearance and spatial layout of the assembly, providing visual cues that are difficult to infer solely from symbolic assembly graphs.

Finally, both the parsed assembly structure and the rendered multi-view images are provided to a vision-language model (VLM). Rather than performing generic image captioning, the VLM acts as an

Table 9: Statistics of AssemBench.

Metric	Value
Assemblies	120
Rendered Image sets	120
GT STEP Models	120
Avg. Bodies	3.38
Avg. Occurrences	4.52
Avg. Joints	2.23
Avg. Contacts	9.43
Avg. Holes	4.78
Avg. B-rep Faces	174.81
Avg. Prompt Length	214 words
Free-form Ratio	0.9%

```

    输入端: {ref: input_gear, port: bore}
    公理: [C-02, P-01]
- ID: m_gear_mesh
  mate_type: gear_mesh
  基础: {ref: input_gear, port: mesh_point}
  输入端: {ref: output_gear, port: mesh_point}
  公理: [G-02]

```

E 失效模式目录

在开发与评估过程中，我们系统地梳理了LLM驱动的CAD装配生成过程中出现的常见故障模式。表8总结了五种最具代表性的故障模式、其根本原因、可观察到的症状，以及Ass-emCAD中用于检测并防止每种故障向最终装配传递的特定验证门控机制。

表8：已识别失效模式的系统化目录，包括各失效模式的根本原因及负责解决相应问题的系统组件。

失效模式	根本原因	症状	门
幻影孔	Workplane("XZ") 标签陷阱会导致 .cut() 无法切中实体。	轴杆穿透未钻孔臂 ($\phi \cdot L$ 碰撞)	港口几何结构。
不相容化合物	联合 (粘合=False) 在...上 面接触	部分会渲染为浮动子部件	安全联合
完全重合部分	零偏移面对面至同一端口	同一位置的多个部件	重叠验证器
端口帧意外事件	基于自动方法构建的 u/v 坐标轴 (位于斜正交面上)	在非预期世界方向上的偏移	严格 x_dir
语义误选	大语言模型为该角色选择了错误的工厂	几何结构正确，但功能上存在错误	视觉反射

F 基准测试构建

F.1 施工管线

Assembench基于Fusion360 Gallery数据集的装配子集构建[23]。我们并未手动标注CAD装配体，而是开发了一套半自动标注流程——该流程结合了基于规则的解析与视觉-语言理解技术，以生成标准化的自然语言装配规范。

给定一个装配实例，我们首先使用基于规则的解析器对其原生Fusion360装配JSON文件进行解析。该解析器可提取结构化的装配信息，包括组件层级关系、装配实例、关节、接触点及其他装配元数据。这种结构化表示方式能够明确描述装配拓扑结构及工程关联关系。

为补充结构信息，每个装配体均利用原始CAD几何数据从两个标准视角进行渲染，所生成的渲染图像可保留整体外观。

以及组件的空间布局，可提供难以仅从符号化组件图中推断出的视觉线索。

最后，解析后的装配结构与渲染出的多视图图像均会提供给视觉-语言模型（VLM）。VLM并非执行通用图像描述任务，而是充当一个具备工程认知能力的标注器，能够联合推

表9：AssemBench的统计结果。

指标	值
装配件	120
渲染图像集	120
GT STEP车型	120
平均尸体数量	3.38
平均发生次数	4.52
平均关节	2.23
平均联系人数量	9.43
平均孔数	4.78
平均B-Rep面	174.81
平均提示词长度与自由格式比例	214 字，占比0.9%

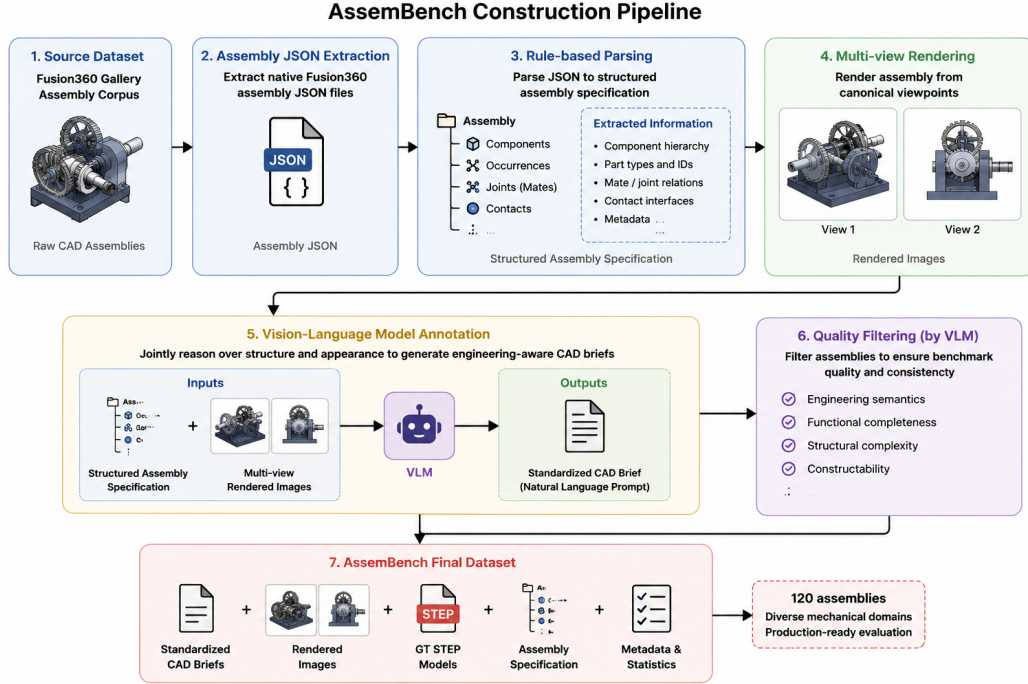


Figure 4: The pipeline of constructing AssemBench

engineering-aware annotator that jointly reasons over geometric appearance and assembly structure to generate standardized CAD briefs. During annotation, the VLM is instructed to preserve engineering semantics, accurately describe functional components and assembly relationships, reflect structural complexity, and avoid unnecessary implementation details. This process produces natural-language specifications that remain faithful to the underlying CAD assemblies while maintaining consistent annotation quality across the benchmark.

Figure 4 illustrates the complete construction pipeline.

F.2 Prompt Generation

The generated CAD briefs are intended to describe *what* should be constructed rather than *how* to implement the corresponding CAD program. Therefore, the prompts emphasize engineering functionality, component interactions, and assembly semantics instead of low-level geometric operations or CAD-specific commands.

To improve consistency, all prompts follow a unified annotation protocol. In particular, the VLM is encouraged to:

- describe the overall mechanical function of the assembly;
- identify major components and their functional roles;
- explain key assembly relationships and engineering semantics;
- preserve the structural complexity of the original assembly;
- avoid implementation-specific details such as CAD operations or programming instructions.

This standardized annotation protocol substantially reduces stylistic variation across prompts while preserving the engineering information required for production-ready CAD assembly generation. The quantitative results are shown in Table 9

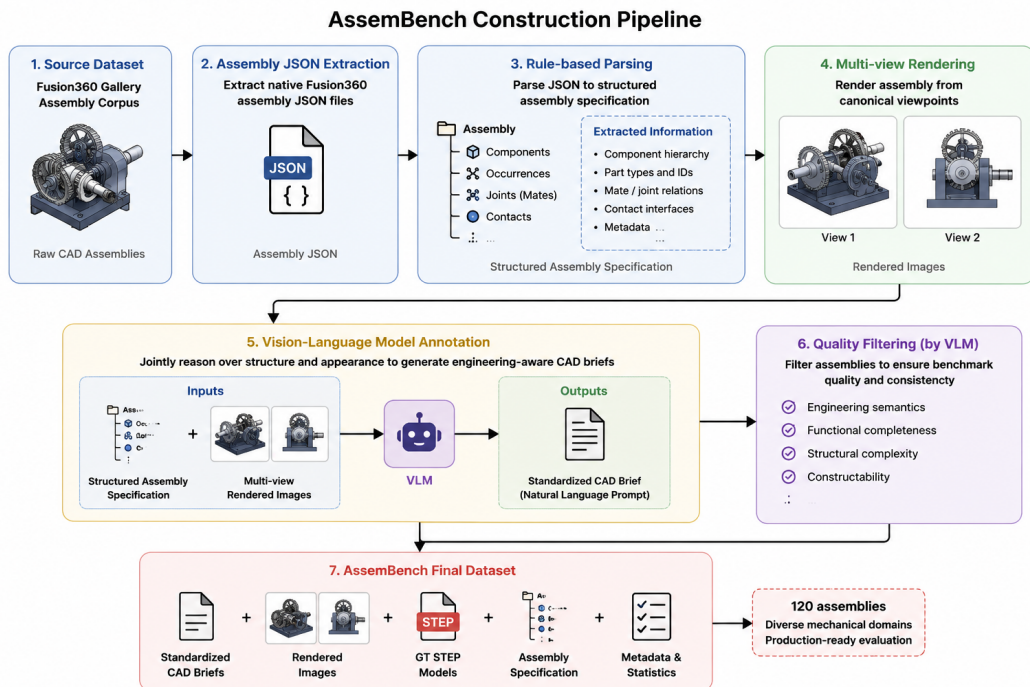


图4: AssemBench构建流程

理几何外观与装配结构，从而生成标准化的CAD技术说明。在标注过程中，VLM 需遵循以下指令：保留工程语义、准确描述功能部件及装配关系、体现结构复杂性，并避免包含不必要的实施细节。这一过程可生成自然语言规格说明，这些说明既忠实于底层CAD装配体，又能在整个基准测试中保持一致的标注质量。

图4展示了完整的构建流程。

F.2 提示词生成

生成的CAD简报旨在描述应构建什么，而非如何实现相应的CAD程序。因此，提示词着重强调工程功能、部件交互及装配语义，而非底层几何运算或CAD专用命令。

为提升一致性，所有提示词均遵循统一的标注规范。具体而言，鼓励 VLM：

- 描述该组件的整体机械功能；
- 识别主要组成部分及其功能作用；
- 阐述关键装配关系与工程语义；
- 保留原始组装的结构复杂性；
- 避免包含实施特定细节，如CAD操作或编程指令。

该标准化标注协议显著减少了不同提示词之间的风格差异，同时保留了生成可直接用于CAD装配的所需工程信息。定量结果如表9所示。

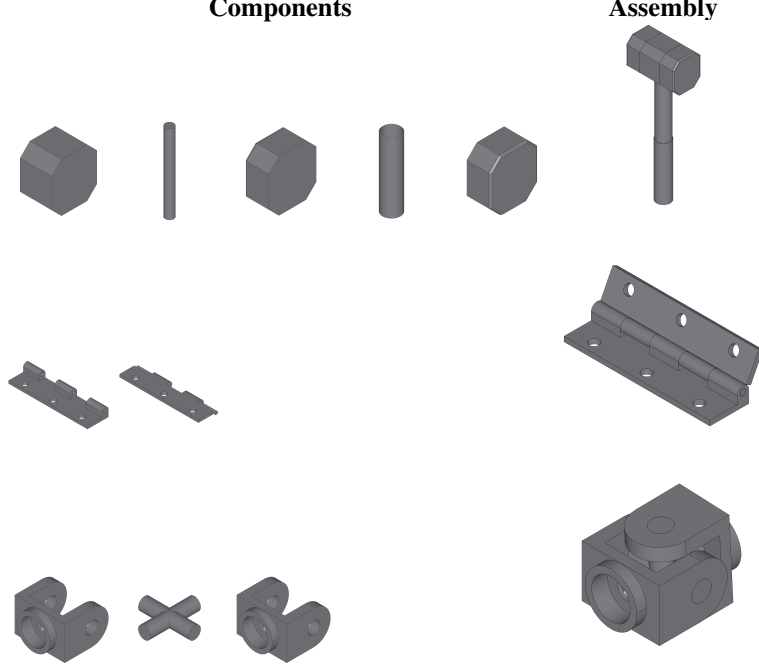


Figure 5: Representative examples from **AssemBench**. Each row shows decomposed CAD components and the corresponding assembled mechanical system.

G Benchmark Visualization

Figure 5 shows representative examples from **AssemBench**. Each example contains decomposed CAD components and the corresponding assembled mechanical system.

H On the Unreliability of Chamfer Distance for Assembly Evaluation

Chamfer Distance (CD) is widely used to evaluate geometric similarity in 3D reconstruction and CAD generation tasks. However, we argue that CD is fundamentally unreliable for evaluating complex mechanical assemblies. This section provides a formal analysis demonstrating two critical deficiencies: CD lacks *assembly structure identifiability* and *assembly ranking consistency*.

H.1 Background and Definition

We first recall the standard definition of Chamfer Distance and describe how it is applied to assembly evaluation.

Definition 5 (Chamfer Distance). *Given two finite point sets $P = \{p_i\}_{i=1}^N \subset \mathbb{R}^3$ and $Q = \{q_j\}_{j=1}^M \subset \mathbb{R}^3$, the squared bidirectional Chamfer Distance is*

$$\text{CD}(P, Q) = \frac{1}{N} \sum_{p \in P} \min_{q \in Q} \|p - q\|_2^2 + \frac{1}{M} \sum_{q \in Q} \min_{p \in P} \|q - p\|_2^2. \quad (4)$$

When applied to assemblies, CD operates on a projected point set $\Phi(\mathcal{A}) = \bigcup_{i=1}^K \text{Sample}(T_i S_i)$, where each part S_i is transformed by $T_i \in SE(3)$. This projection discards part identity, assembly graph structure, contact relationships, kinematic constraints, and physical feasibility.

H.2 Information Loss under Point-Set Projection

The fundamental issue with applying CD to assemblies is the information loss incurred by projecting a structured assembly into an unordered point set. The following lemma formalizes this observation.

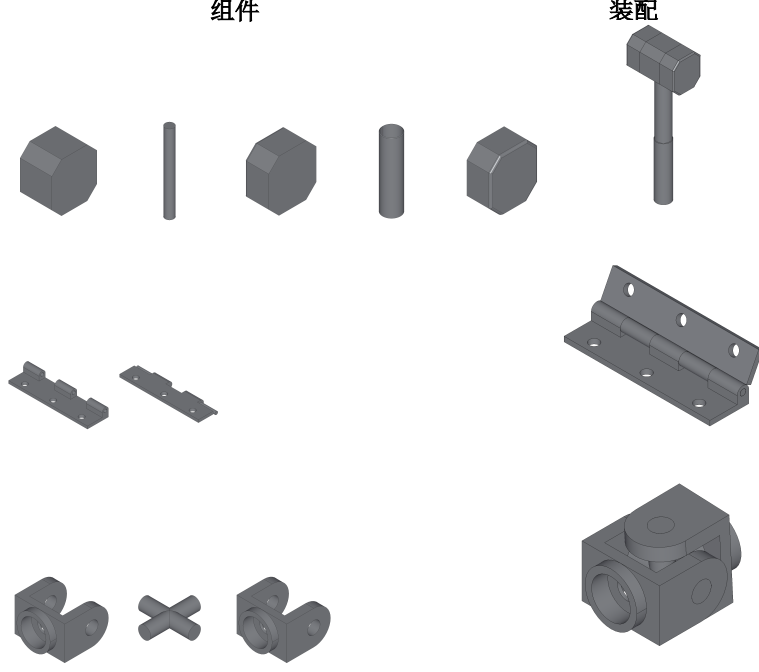


图5: **AssemBench**的代表性示例。每行展示了解析后的CAD组件及对应的装配机械系统。

G 基准测试可视化

图5展示了来自**AssemBench**的代表性示例。每个示例均包含分解后的CAD组件及对应的装配机械系统。

H: 关于倒角距离在装配评估中的不可靠性

倒角距离（CD）被广泛用于评估三维重建和CAD生成任务中的几何相似性。然而，我们认为CD在评估复杂的机械装配时本质上是不可靠的。本节通过形式化分析揭示了其两个关键缺陷：CD缺乏装配结构可识别性与装配排序一致性。

H.1 背景与定义

我们首先回顾 Chamfer Distance 的标准定义，并阐述其在装配评估中的应用方式。

定义5（Chamfer距离）。给定两个有限点集 $P = \{p_i\}_{i=1}^N \subset \mathbb{R}^3$ 且 $Q = \{q_j\}_{j=1}^M \subset \mathbb{R}^3$ ，平方双向倒角距离为

$$\text{CD}(P, Q) = \frac{1}{N} \sum_{p \in P} \min_{q \in Q} \|p - q\|_2^2 + \frac{1}{M} \sum_{q \in Q} \min_{p \in P} \|q - p\|_2^2. \quad (4)$$

当应用于装配时，CD算法作用于投影点集 $\Phi(\mathcal{A}_{\text{sp}_1}) = \mathcal{U}$ 。 $\sum_{i=1}^K$ 样本 $(T_i S_i)$ ，其中每个部件 S_i 均通过 $T_i \in SE(3)$ 进行变换。此投影过程会舍弃部件标识、装配图结构、接触关系、运动学约束及物理可行性。

H.2 点集投影下的信息损失

将CD方法应用于组装过程的根本问题在于：将结构化组装投影为无序点集时会引发信息损失。以下引理可对这一观察结果进行形式化表述。

Lemma 1 (Irrecoverability of Assembly Structure). *Let f be any evaluation function that depends only on point-set projections, i.e., $f(\mathcal{A}, \mathcal{A}^*) = g(\Phi(\mathcal{A}), \Phi(\mathcal{A}^*))$ for some function g . If two assemblies $\mathcal{A}_1, \mathcal{A}_2$ satisfy $\Phi(\mathcal{A}_1) = \Phi(\mathcal{A}_2)$, then $f(\mathcal{A}_1, \mathcal{A}^*) = f(\mathcal{A}_2, \mathcal{A}^*)$ for any reference $\mathcal{A}^*$.*

Proof. By definition, $f(\mathcal{A}_1, \mathcal{A}^*) = g(\Phi(\mathcal{A}_1), \Phi(\mathcal{A}^*)) = g(\Phi(\mathcal{A}_2), \Phi(\mathcal{A}^*)) = f(\mathcal{A}_2, \mathcal{A}^*)$. $\square$

Since CD is precisely such a function, it inherits this limitation: any assembly information lost during point-set projection is invisible to CD.

H.3 CD Lacks Assembly Structure Identifiability

Building on the irrecoverability result, we show that CD cannot distinguish assemblies that differ in structurally important ways, such as part count, assembly graph topology, or constraint relationships whenever their surface point sets coincide.

Proposition 3 (Non-Identifiability). *There exist assemblies $\mathcal{A}_1$ and $\mathcal{A}_2$ that differ in part count, assembly graph, or constraint relationships, yet satisfy $\Phi(\mathcal{A}_1) = \Phi(\mathcal{A}_2)$, and hence $\text{CD}(\Phi(\mathcal{A}_1), \Phi(\mathcal{A}^*)) = \text{CD}(\Phi(\mathcal{A}_2), \Phi(\mathcal{A}^*))$ for any reference $\mathcal{A}^*$.*

Proof. Consider a two-part assembly $\mathcal{A}_1 = (\{S_1, S_2\}, \{T_1, T_2\}, \mathcal{G}_1, \mathcal{C}_1)$. Construct $\mathcal{A}_2$ as a single fused part $S' = T_1 S_1 \cup T_2 S_2$ with identity pose. Although $\mathcal{A}_1$ and $\mathcal{A}_2$ differ in part count, assembly graph, and disassembly relationships, their surface point sets are identical: $\Phi(\mathcal{A}_1) = \Phi(\mathcal{A}_2)$. By Lemma 1, CD cannot distinguish them. $\square$

This result extends to part identity swaps, left-right mirror errors, and missing constraint relationships—any structural defect that preserves the overall surface geometry remains invisible to CD.

H.4 CD Lacks Assembly Ranking Consistency

Beyond identifiability, a reliable assembly metric should rank a structurally superior prediction closer to the reference. We formalize this property and prove that CD violates it: a structurally complete assembly can receive a worse CD score than one missing a functionally critical component.

Definition 6 (Assembly Ranking Consistency). *A metric d is assembly-ranking-consistent if, whenever $Q_{\text{asm}}(\mathcal{A}_1, \mathcal{A}^*) < Q_{\text{asm}}(\mathcal{A}_2, \mathcal{A}^*)$ for an ideal assembly quality function Q_{asm} , it follows that $d(\mathcal{A}_1, \mathcal{A}^*) < d(\mathcal{A}_2, \mathcal{A}^*)$.*

Theorem 1 (Ranking Reversal). *CD is not assembly-ranking-consistent. There exist a reference P^* and two predictions $P_{\text{good}}, P_{\text{bad}}$ such that P_{good} is structurally superior yet $\text{CD}(P_{\text{good}}, P^*) > \text{CD}(P_{\text{bad}}, P^*)$.*

Proof. Let the reference consist of a large part with N sample points P_L and a small but functionally critical part with m sample points P_S , so $P^* = P_L \cup P_S$ with $N \gg m$. Let $\bar{D}^2 = \frac{1}{m} \sum_{s \in P_S} \min_{l \in P_L} \|s - l\|_2^2$ denote the average squared nearest-neighbor distance from the small part to the large part.

Construct two predictions:

- $P_{\text{good}} = (P_L + \epsilon \mathbf{u}) \cup P_S$: all parts present, large part shifted by ϵ .
- $P_{\text{bad}} = P_L$: large part perfectly aligned, small critical part missing.

Direct computation yields

$$\text{CD}(P_{\text{good}}, P^*) \approx \frac{2N\epsilon^2}{N+m}, \quad \text{CD}(P_{\text{bad}}, P^*) = \frac{m\bar{D}^2}{N+m}.$$

Ranking reversal occurs when $2N\epsilon^2 > m\bar{D}^2$, i.e.,

$$N > \frac{m\bar{D}^2}{2\epsilon^2}. \quad (5)$$

Since large structural parts (housings, frames, base plates) typically have $N \gg m$ relative to small connectors (bolts, pins, bearings), this condition is easily satisfied in practice. Thus a structurally complete prediction receives a worse CD score than one missing a functionally critical component. $\square$

引理1（装配结构不可恢复性）：设 f 为任意仅依赖于点集投影的评估函数，即 $f(A, A^*) = g(\Phi(A), \Phi(A^*))$ ，其中 g 为某函数。若两个装配体 A_1, A_2 满足 $\Phi(A_1) = \Phi(A_2)$ ，则对于任意参考装配体 A^* ，有 $f(A_1, A^*) = f(A_2, A^*)$ 。

证明：根据定义， $f(A_1, A^*) = g(\Phi(A_1), \Phi(A^*)) = g(\Phi(A_2), \Phi(A^*)) = f(A_2, A^*)$ 。□

由于CD正是此类函数，它继承了这一局限性：在点集投影过程中丢失的任何装配信息对CD而言均不可见。

H.3 CD缺乏装配结构可辨识性

基于不可恢复性结果，我们证明：当不同组装体的表面点集相同时，CD无法区分那些在结构上具有重要差异（例如部件数量、组装图拓扑结构或约束关系）的组装体。

命题3（不可辨识性）：存在装配体 A_1 与 A_2 ，它们在零件数量、装配图或约束关系方面存在差异，但仍满足 $\Phi(A_1) = \Phi(A_2)$ ，因此对于任意参考装配体 A^* ，有 $CD(\Phi(A_1), \Phi(A^*)) = CD(\Phi(A_2), \Phi(A^*))$ 。

证明：考虑一个两部分装配结构 $A_1 = (\{S_1, S_2\}, \{T_1, T_2\}, G_1, C_1)$ 。构造 A_2 为一个单一融合部件 $S = T_1 \cup T_2$ ，其初始姿态为恒等姿态。尽管 A_1 与 A_2 在部件数量、装配图及拆卸关系上存在差异，但它们的表面点集完全相同： $\Phi(A_1) = \Phi(A_2)$ 。根据引理1，CD无法区分二者。□

该结果可推广至部分结构交换、左右镜像错误及约束关系缺失等情况——任何能保持整体表面几何结构的结构缺陷，均不会被CD检测到。

H.4 CD的装配排名一致性不足

除可识别性之外，可靠的组装度量应能将结构上更优的预测结果排在更接近参考值的位置。我们对这一性质进行了形式化定义，并证明了CD准则违反该性质：一个结构完整的组装可能获得比缺失功能关键组件的组装更差的CD评分。**定义6**（组装排序一致性）：若度量 D 满足以下条件，则称其具有组装排序一致性： $Q_{asm}(A_1) < Q_{asm}(A_2)$ 对于某个理想组装质量函数 Q_{asm} 成立时，则有 $D(A_1) < D(A_2)$ 。

定理1（排序逆转）：CD并非组装排序一致性。存在一个参考 P^* 以及两个预测 P_{good}, P_{bad} ，使得 P_{good} 在结构上更优，但 $CD(P_{good}, P^*) > CD(P_{bad}, P^*)$ 。

证明：设参考集由包含 N 个样本点的 P_L 的大部分区域和包含 m 个样本点的 P_S 的小但功能关键的部分组成，即 $P^* = P_L \cup P_S$ with $N \gg m$ 。令 $\bar{D}^2 = \frac{1}{m} \sum_{s \in P_S} \min_{l \in P_L} \|s - l\|^2$ 表示从小部分到大部分的最近邻距离平方。

构建两个预测：

- $P_{good} = (P_L + \epsilon \mathbf{u}) \cup P_S$: all parts present, large part shifted by ϵ .
- $P_{bad} = P_L$: large part perfectly aligned, small critical part missing.

直接计算可得：

$$CD(P_{good}, P^*) \approx \frac{2N\epsilon^2}{N+m}, \quad CD(P_{bad}, P^*) = \frac{m\bar{D}^2}{N+m}.$$

当 $2N\epsilon^2 > m\bar{D}^2$ 时，会发生排名反转，即：

$$\frac{m\bar{D}^2}{2\epsilon^2} > N. \quad (5)$$

由于大型结构部件（外壳、框架、底板）与小型连接件（螺栓、销钉、轴承）之间的相对关系通常满足 $N \gg m$ ，因此在实际应用中，这一条件极易满足。因此，一个结构完整的预测其CD评分会低于缺失了功能关键组件的预测结果。□

H.5 Surface-Area Bias

Under uniform surface sampling, the number of points from part i satisfies $N_i \approx N_{\text{total}} \cdot A_i/A_{\text{total}}$. Therefore, global CD implicitly weights each part's error by its surface area fraction:

$$\text{CD}_{\text{global}} \approx \sum_{i=1}^K \frac{A_i}{A_{\text{total}}} e_i,$$

where e_i is the per-part average squared error. Large-surface-area components dominate the score regardless of functional importance, while small but mechanically critical parts (fasteners, bearings, shafts) contribute negligibly.

H.5 表面积偏差

在均匀表面采样下，来自部件*i*的采样点数量满足 $N_i \approx N_{\text{total}} \cdot A_i / A_{\text{total}}$ 。因此，全局CD方法会通过各部件的表面积占比来隐式地对各部件的误差进行加权：

$$\text{CD}_{\text{global}} \approx \sum_{i=1}^K \frac{A_i}{A_{\text{total}}} e_i,$$

其中 e_i 表示各部件的平均平方误差；无论功能重要性如何，表面积较大的部件均主导评分结果，而体积虽小但对机械性能至关重要的部件（如紧固件、轴承、轴）所贡献的值可忽略不计。